Título del TFM

Nombre del autor

September 7, 2026

Contents

1 Basic operation **3**

1.1 Screens and settings . . . 3

1.1.1 Main screen . . . 3

1.1.2 Jog screen . . . 5

1.1.3 Profile screen . . . 7

1.1.4 Optical alignment screen . . . 8

2 Clases de la API **12**

2.1 Clase System . . . 12

2.2 Clase AxisComponents . . . 13

2.3 Clase Axis2D . . . 14

2.4 Clase Axis3D . . . 15

2.5 Clase Alignment . . . 17

2.6 Clase Profile . . . 20

2.7 Clase Angle Asjustment . . . 21

3 Routines **23**

3.1 RasterScan . . . 23

3.1.1 `clean_profile_data()` . . . 24

3.1.2 `save_profile_txt()` . . . 25

3.1.3 `create_heatmap()` . . . 26

3.2 Automaper . . . 28

3.2.1 Building blocks . . . 28

3.2.2 checkPositionsAutomap . . . 29

3.2.3 moveAutomapOPM . . . 30

3.2.4 Workflow . . . 30

3.2.5 Relevant Configuration . . . 30

3.2.6 Results . . . 31

3.2.7 Precautions . . . 31

3.2.8 moveAutomapOPM_OSA 31
3.2.9 Workflow 31
3.2.10 Relevant Configuration 31
3.2.11 Results 32
3.2.12 Precautions 32

A Automap **33**

B Instalación TwinCAT XAE eXtended Automation Engineering **35**

C Documentacion de la API Suruga Seiki **41**
C.1 Clase Alignment 41
C.2 Clase AngleAdjustment 48
C.3 Clase Axis2D 49
C.4 Clase Axis3D 50
C.5 Clase AxisComponents 51
C.6 Clase ErrorLog 51
C.7 Clase IO 52
C.8 Clase PowerMeter 52
C.9 Clase System 53

Chapter 1

Basic operation

1.1 Screens and settings

1.1.1 Main screen

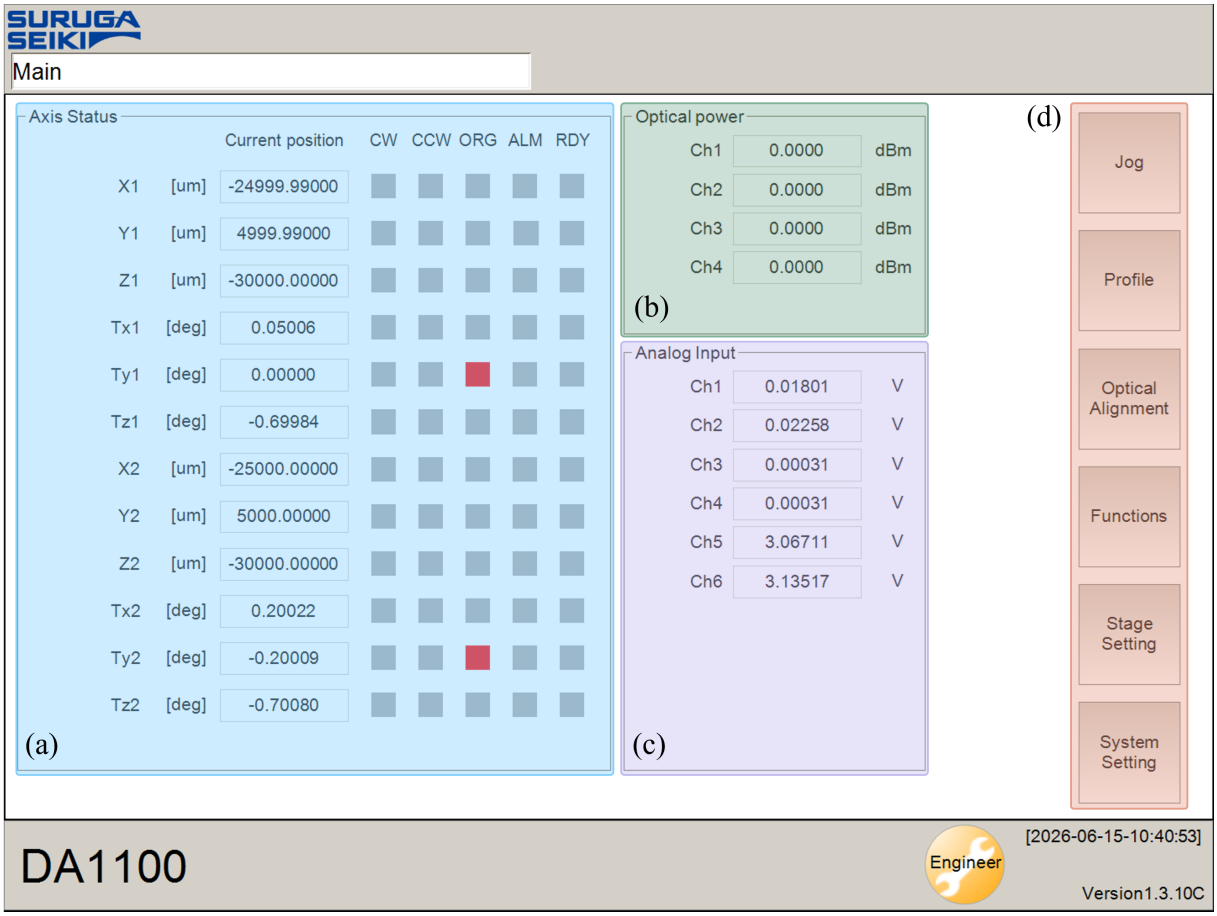


Figure 1.1: Main screen of the Suruga positioning system. The interface is divided into four main sections: (a) Axis Status, which displays the current position and status of each axis; (b) Optical Power, which displays the optical power measured by the connected power meter; (c) Analog Input, which displays the voltage measured at each available analog input channel; and (d) Function Access, which provides direct access to the main functionalities and configuration options of the Suruga platform.

This screen is divided into three main sections. The first section, Axis Status, allows the user to monitor the current position of each axis, the CW and CCW limit switch status, the origin sensor status, the alarm outputs, and the availability of each servo. The second section, Optical Power, displays the current reading of the power meter, if one is available. Finally, the Analog Input section allows the user to monitor the values of the different analog inputs. On the left side of the screen, direct access to the different functionalities of the Suruga platform is provided.

- **Current Position:** Displays the current coordinates of each axis. Linear axes are displayed in micrometers (μm), while rotational axes are displayed in degrees ($^\circ$). Axes with the number 1 correspond to the left arm of the Suruga positioning system, while axes with the number 2 correspond to the right arm.
- **CW:** Displays the status of the clockwise (CW) limit sensor. Red indicates that the limit has been reached, while gray indicates that the axis is within the safe operating range.
- **CCW:** Displays the status of the counterclockwise (CCW) limit sensor. Red indicates that the limit has been reached, while gray indicates that the axis is within the safe operating range.
- **ORG:** Displays the status of the origin sensor. For this particular stage, no origin sensor is available, so the indicator will remain continuously displayed in red or gray.
- **ALM:** Indicates the alarm status of the axis. Red is displayed when an alarm occurs, such as an overload or a similar fault condition. Gray indicates normal operation.
- **RDY:** Displays the operation status of each axis. Green indicates that the servo is enabled and ready for operation, while gray indicates that the servo is turned off.

1.1.2 Jog screen

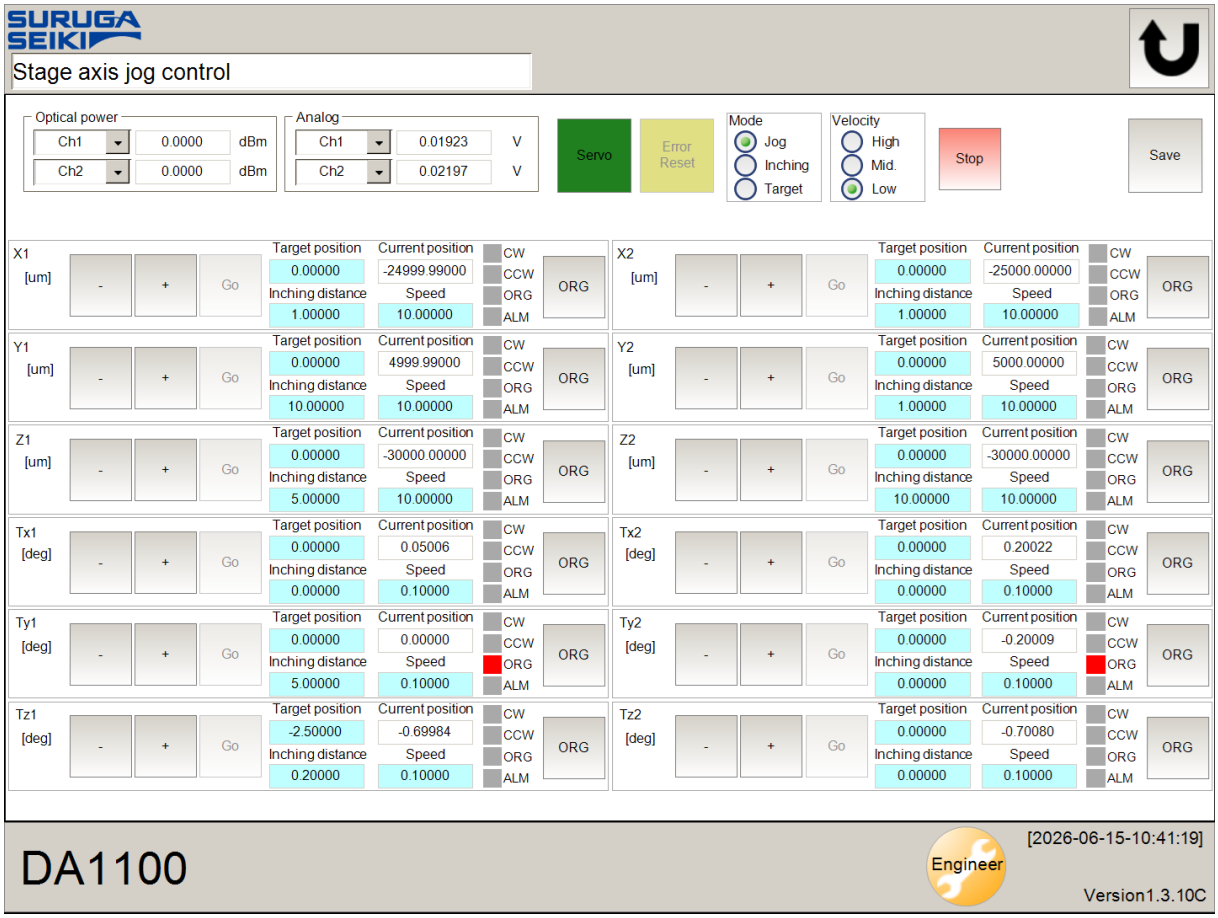


Figure 1.2: Pendiente.

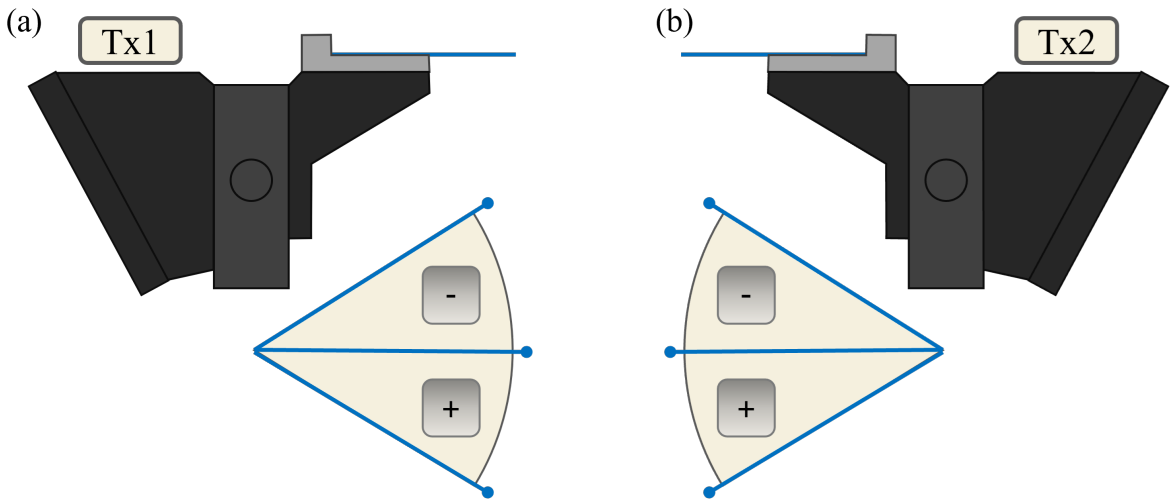


Figure 1.3: Pendiente.

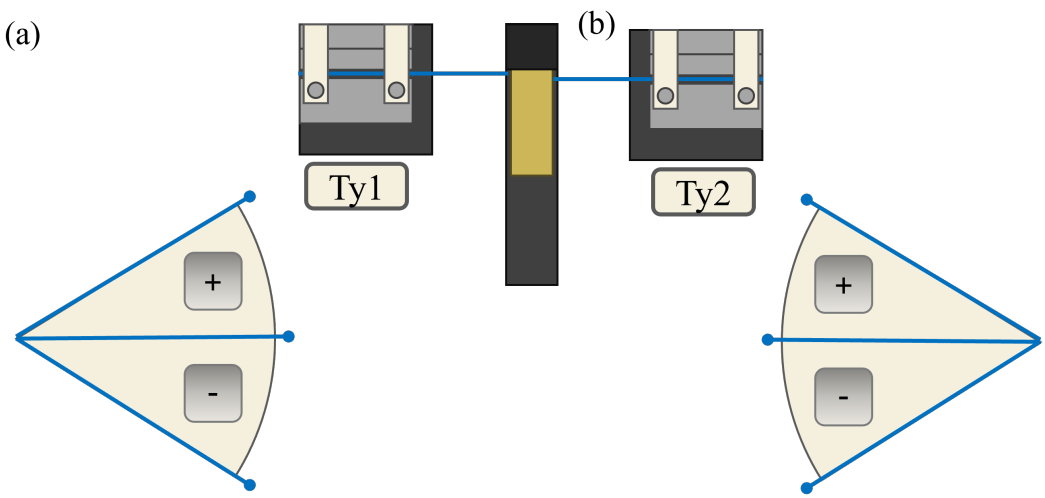


Figure 1.4: Pendiente.

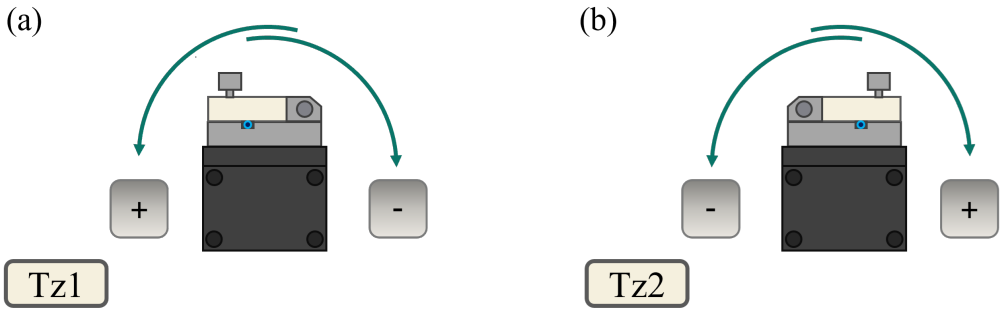


Figure 1.5: Pendiente.

1.1.3 Profile screen



Figure 1.6: Pendiente.

1.1.4 Optical alignment screen

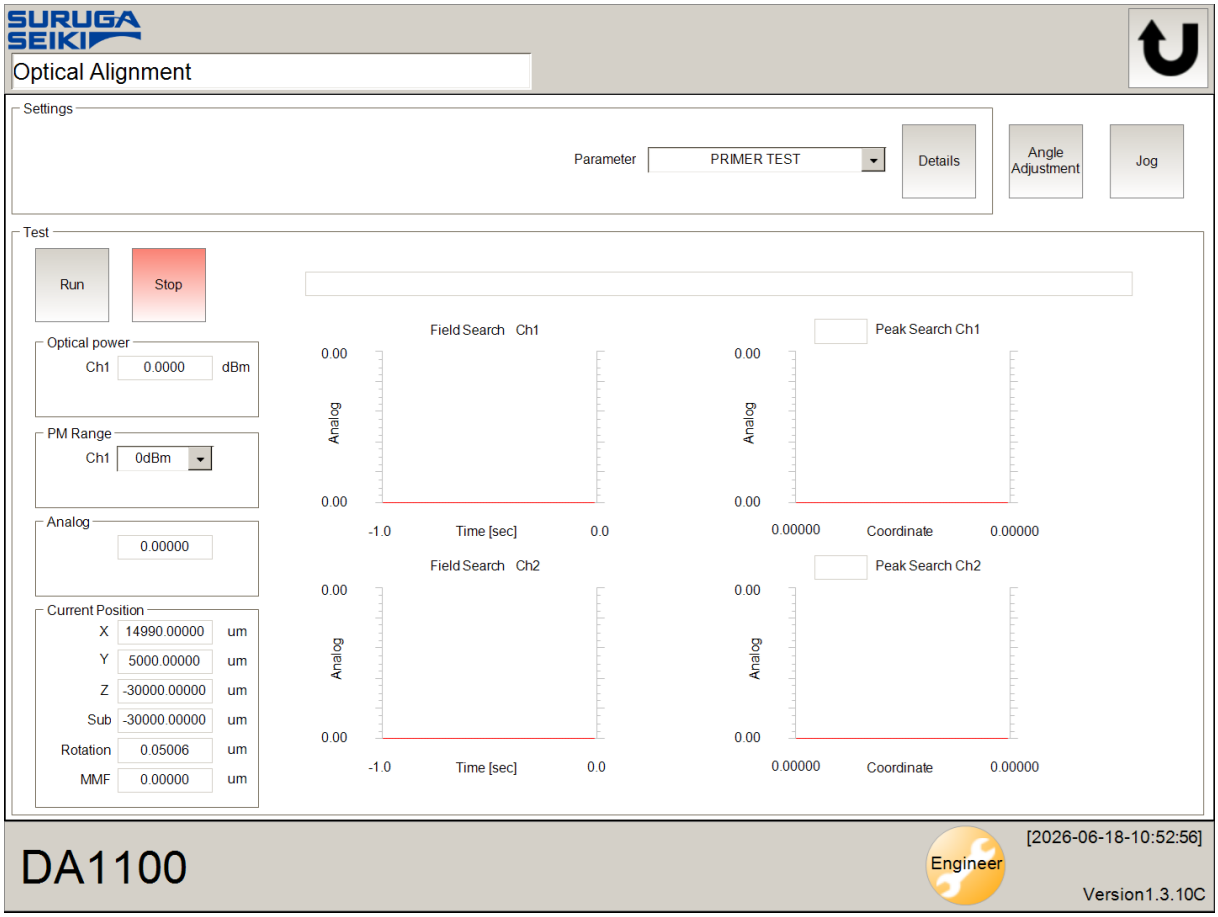


Figure 1.7: Pendiente.

El usuario puede llevar a cabo los siguientes métodos de alineamiento:

- **Alineamiento Single (Single Alignment):** Realiza el alineamiento utilizando un único eje.
- **Alineamiento Flat (Flat Alignment):** Realiza el alineamiento en un plano utilizando dos ejes.
- **Alineamiento Focus (Focus Alignment):** Además del alineamiento Flat, realiza el ajuste sobre el eje óptico empleando tres ejes.
- **Alineamiento por rotación de dos canales (2ch Rotation Alignment):** Realiza un alineamiento Flat utilizando dos señales ópticas y corrige el ángulo alrededor del eje óptico en función de la posición y de la distancia configurada.
- **Alineamiento MMF (MMF Rotation Alignment):** Posiciona una fibra multimodo (MMF, Multi-Mode Fiber) en el lado de salida y corrige la posición y el ángulo del arreglo de fibras en el lado de entrada.

Alineamiento Flat (Flat Alignment):

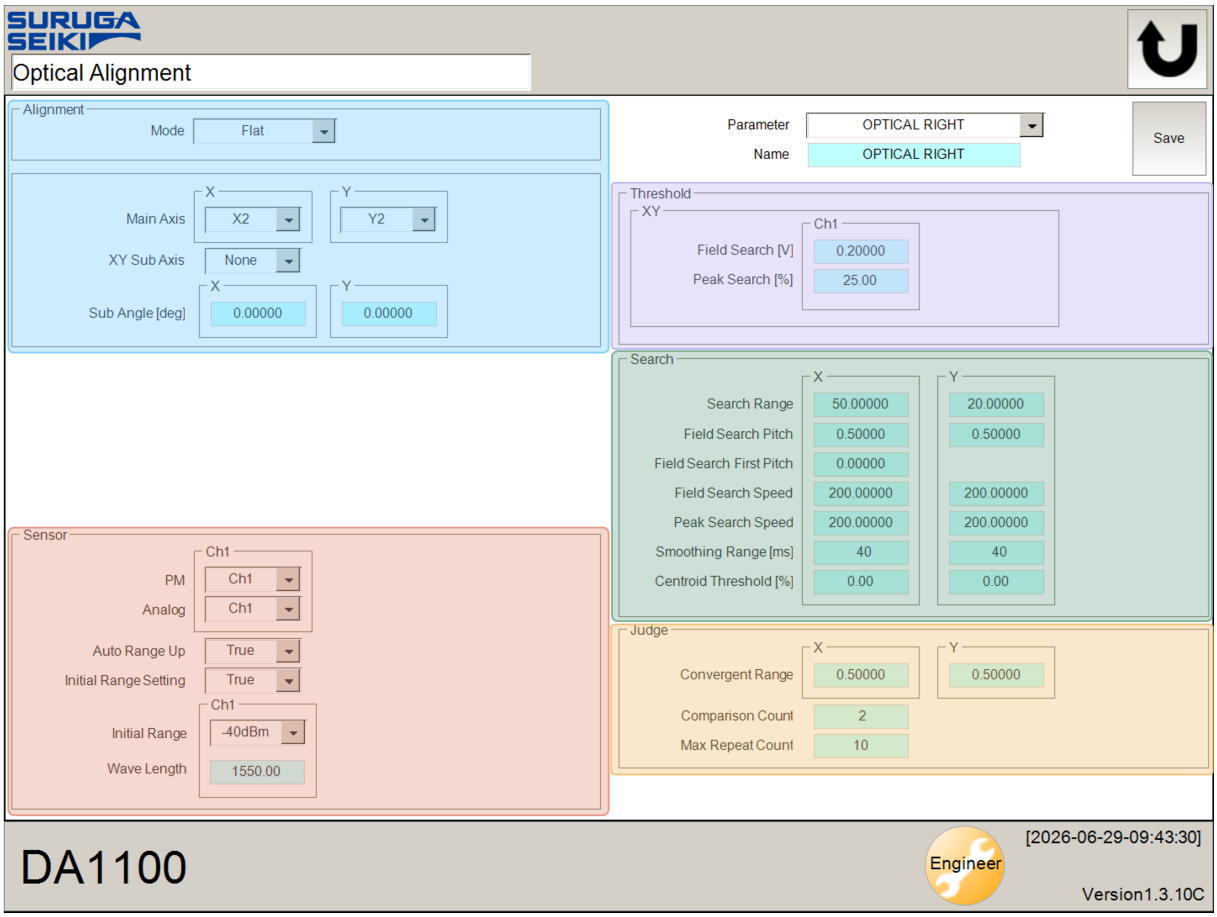


Figure 1.8: Pendiente.

Se parte de la condición en la que la fibra y el Device Under Test (DUT) se encuentran desalineados, por lo que el sistema desconoce la ubicación del punto de máxima potencia óptica. Debido a ello, no es posible iniciar directamente un Peak Search, ya que este requiere partir de una señal detectable para localizar el máximo.

Como primer paso, el algoritmo ejecuta un Field Search, cuyo objetivo es explorar el área alrededor de la posición inicial hasta encontrar una potencia lo suficientemente grande. La condición que determina el éxito de esta etapa es:

$$P_{\text{OPM}} > \text{FieldSearchThreshold} \tag{1.1}$$

Donde FieldSearchThreshold corresponde al nivel mínimo de potencia, expresada en voltios haciendo uso de la salida analógica del OPM, que indica que el sistema ha encontrado una señal válida.

Durante esta etapa, el movimiento sigue una trayectoria en espiral cuadrada que se expande progresivamente alrededor del punto inicial. En cuanto la potencia medida supera el umbral establecido, el algoritmo abandona el Field Search e inicia el proceso de Peak Search, encargado de localizar con precisión el punto de máxima potencia.

Si la trayectoria alcanza los límites definidos por SearchRangeX o SearchRangeY sin encontrar una señal superior a FieldSearchThreshold, el proceso finaliza con el estado FieldSearchRangeOver, indicando que no fue posible localizar una señal dentro del área de búsqueda especificada.

El comportamiento del algoritmo empleado para un alineamiento tipo flat está determinado por un conjunto de parámetros que definen el área de búsqueda, la resolución del barrido y el criterio para iniciar la búsqueda fina. En la Tabla 1.1 se describen los parámetros más relevantes en conjunto con su equivalente en el DLL provisto por Suruga Seiki.

Table 1.1: Parámetros de configuración

Parámetro	**Descripción**	**Tipo .NET**
Alignment		
mainStageNumberX	Número del eje principal utilizado para el movimiento en X.	UInt16
mainStageNumberY	Número del eje principal utilizado para el movimiento en Y.	UInt16
subStageNumberXY	Número del sub-eje utilizado para el movimiento en XY.	UInt16
subAngleX	Ángulo del sub-eje para la alineación en X.	Double
subAngleY	Ángulo del sub-eje para la alineación en Y.	Double
Sensor		
pmCh	Canal del medidor de potencia utilizado durante la alineación.	UInt16
analogCh	Canal de entrada analógica utilizado para la medición.	UInt16
pmAutoRangeUpOn	Habilita el incremento automático del rango del medidor de potencia.	Boolean
pmInitRangeSettingOn	Habilita el uso de un rango inicial para el medidor de potencia.	Boolean
pmInitRange	Valor del rango inicial del medidor de potencia.	Int16
Threshold		
fieldSearchThreshold	Umbral utilizado durante la búsqueda de campo.	Double
peakSearchThreshold	Umbral utilizado durante la búsqueda del pico de señal.	Double
Search		
searchRangeX	Rango de búsqueda en la dirección X.	Double
searchRangeY	Rango de búsqueda en la dirección Y.	Double
fieldSearchPitchX	Paso de exploración en X durante la búsqueda de campo.	Double
fieldSearchPitchY	Paso de exploración en Y durante la búsqueda de campo.	Double
fieldSearchFirstPitchX	Primer paso de exploración en X durante la búsqueda de campo.	Double
fieldSearchSpeedX	Velocidad de exploración en X durante la búsqueda de campo.	Double

Continued on next page

Table 1.1: Parámetros de configuración (Continued)

fieldSearchSpeedY	Velocidad de exploración en Y durante la búsqueda de campo.	Double
peakSearchSpeedX	Velocidad de búsqueda del pico en X.	Double
peakSearchSpeedY	Velocidad de búsqueda del pico en Y.	Double
smoothingRangeX	Ventana de suavizado aplicada a los datos en X.	UInt32
smoothingRangeY	Ventana de suavizado aplicada a los datos en Y.	UInt32
centroidThresholdX	Umbral para el cálculo del centroide en X.	Double
centroidThresholdY	Umbral para el cálculo del centroide en Y.	Double
Judge		
convergentRangeX	Rango de convergencia permitido en X.	Double
convergentRangeY	Rango de convergencia permitido en Y.	Double
comparisonCount	Número de comparaciones realizadas para verificar la convergencia.	UInt16
maxRepeatCount	Número máximo de repeticiones permitidas durante el proceso de alineación.	UInt16

Chapter 2

Clases de la API

2.1 Clase System

La clase `System` constituye el punto de acceso principal a las funciones globales del controlador Suruga Seiki. A través de esta clase es posible consultar las capacidades del sistema, verificar el estado de la comunicación con el controlador, obtener información sobre la versión del software y gestionar operaciones generales relacionadas con la conexión y el diagnóstico del equipo.

La Tabla 2.1 resume los principales miembros públicos de la clase, agrupados en propiedades, métodos, campos y tipos anidados. Para cada elemento se indica su propósito, el tipo de dato asociado y el nivel de acceso proporcionado por la API.

Table 2.1: Miembros de la clase `System`

Miembro	Descripción	Tipo .NET	Acceso
Propiedades			
MaxAxisStageNumber	Número máximo de ejes de posicionamiento soportados por el sistema.	UInt16	get/set
MaxAnalogInputChNumber	Número máximo de canales de entrada analógica disponibles.	UInt16	get/set
MaxAlignmentNumber	Número máximo de instancias de alineación soportadas por el sistema.	UInt16	get/set
MaxAngleAdjustmentNumber	Número máximo de módulos de ajuste angular soportados.	UInt16	get/set
MaxDInChNumber	Número máximo de canales de entrada digital disponibles.	UInt16	get/set
MaxDOutChNumber	Número máximo de canales de salida digital disponibles.	UInt16	get/set
MaxAnalogOutputChNumber	Número máximo de canales de salida analógica disponibles.	UInt16	get/set
MaxPowerMeterChNumber	Número máximo de canales del medidor de potencia soportados.	UInt16	get/set
Instance	Instancia única (Singleton) de la clase `System`.	System	get
Connected	Indica si existe una conexión activa con el controlador.	Boolean	get
DllVersion	Versión de la biblioteca dinámica (DLL) utilizada por la API.	String	get

Continued on next page

Table 2.1: Miembros de la clase `System` (Continued)

SystemVersion	Versión del firmware o software del sistema de control.	String	get
Métodos			
IsError()	Determina si el sistema presenta algún estado de error.	Boolean	–
GetCurrentErrorCode()	Obtiene el código del error actual del sistema.	ErrorCode	–
IsEmergencyAsserted()	Indica si la condición de parada de emergencia está activada.	Boolean	–
SetAddress(String addr)	Establece la dirección del controlador utilizada para la comunicación.	Void	–
DisconnectAds()	Finaliza la conexión ADS con el controlador.	Void	–
Campos			
MaxProfileNumber	Número máximo de perfiles soportados por el sistema.	UInt16	static
ProfileDataPacketMaxDataNumbe	Número máximo de datos que puede contener un paquete de datos de perfil.	UInt32	static
Tipos anidados / Enums			
ErrorCode	Enumeración que define los posibles códigos de error devueltos por la clase `System`.	Enum	–

2.2 Clase AxisComponents

Table 2.2: Miembros de la clase `AxisComponents`

Miembro	**Descripción**	**Tipo .NET**	**Acceso**
Constructores			
AxisComponents(UInt16 axisNumber)	Inicializa una nueva instancia de la clase asociada al eje especificado.	Constructor	–
Métodos			
MoveAbsolute(Double position)	Desplaza el eje hasta una posición absoluta.	ErrorCode	–
MoveRelative(Double distance)	Desplaza el eje una distancia relativa respecto a su posición actual.	ErrorCode	–
ReturnOrigin()	Ejecuta el procedimiento de retorno al origen del eje.	ErrorCode	–
Stop()	Detiene inmediatamente el movimiento del eje.	ErrorCode	–
TurnOnServo()	Activa el servomotor del eje.	ErrorCode	–
TurnOffServo()	Desactiva el servomotor del eje.	ErrorCode	–
ResetError()	Restablece el estado de error del eje.	ErrorCode	–
GetTargetPosition()	Obtiene la posición objetivo configurada para el eje.	Double	–
GetActualPosition()	Obtiene la posición actual del eje.	Double	–
IsServoOn()	Indica si el servomotor se encuentra activado.	Boolean	–
IsServoAlarm()	Indica si el servomotor presenta una condición de alarma.	Boolean	–

Continued on next page

Table 2.2: Miembros de la clase `AxisComponents` (Continued)

IsMoving()	Indica si el eje se encuentra en movimiento.	Boolean	–
IsOriginSignalOn()	Indica si la señal del sensor de origen está activa.	Boolean	–
IsCwLimitSignalOn()	Indica si el interruptor de límite en sentido horario (CW) está activo.	Boolean	–
IsCcwLimitSignalOn()	Indica si el interruptor de límite en sentido antihorario (CCW) está activo.	Boolean	–
IsCwTorqueLimitOn()	Indica si se ha alcanzado el límite de par en sentido horario.	Boolean	–
IsCcwTorqueLimitOn()	Indica si se ha alcanzado el límite de par en sentido antihorario.	Boolean	–
IsOriginReturnSucceeded()	Indica si el procedimiento de retorno al origen finalizó correctamente.	Boolean	–
SetAccelRate(Double rate)	Configura la tasa de aceleración del eje.	ErrorCode	–
SetDecelRate(Double rate)	Configura la tasa de desaceleración del eje.	ErrorCode	–
SetSineMotion(Boolean sineMotion)	Habilita o deshabilita el modo de movimiento senoidal.	ErrorCode	–
SetFulcrumSpan(Double span)	Configura la distancia del brazo o fulcro utilizada durante el movimiento.	ErrorCode	–
SetEncoderStepLength(Double length)	Configura la longitud correspondiente a un paso del codificador.	ErrorCode	–
SetMaxSpeed(Double speed)	Establece la velocidad máxima permitida para el eje.	ErrorCode	–
SetCWTorqueLimit(Double cWlimit)	Configura el límite de par en sentido horario (CW).	ErrorCode	–
SetCCWTorqueLimit(Double cCWlimit)	Configura el límite de par en sentido antihorario (CCW).	ErrorCode	–
GetErrorCode()	Obtiene el código de error actual del eje.	ErrorCode	–
GetStatus()	Obtiene el estado actual del eje.	Status	–
Tipos anidados / Enums			
ErrorCode	Enumeración que define los posibles códigos de error devueltos por la clase `AxisComponents`.	Enum	–
Status	Enumeración que representa los distintos estados de funcionamiento del eje.	Enum	–

2.3 Clase Axis2D

Table 2.3: Miembros de la clase `Axis2D`

Miembro	Descripción	Tipo .NET	Acceso
Constructores			
Axis2D(UInt16 mainStageNumber, UInt16 subStageNumber)	Inicializa una nueva instancia de la clase asociada a un sistema de posicionamiento bidimensional compuesto por dos ejes.	Constructor	–
Métodos			
SetAxisNumber(UInt16 mainStageNumber, UInt16 subStageNumber)	Asigna los identificadores de los ejes principal y secundario utilizados por el sistema bidimensional.	ErrorCode	–

Continued on next page

Table 2.3: Miembros de la clase `Axis2D` (Continued)

GetAxisNumber()	Obtiene los identificadores de los ejes asociados al sistema bidimensional.	UInt16[]	–
MoveAbsolute(Point targetPosition)	Desplaza ambos ejes hasta una posición absoluta especificada mediante una estructura de coordenadas.	ErrorCode	–
MoveAbsolute(Double mainTargetPosition, Double subTargetPosition)	Desplaza ambos ejes hasta las posiciones absolutas indicadas para cada eje.	ErrorCode	–
MoveRelative(Point targetDistance)	Realiza un desplazamiento relativo de ambos ejes utilizando una estructura de coordenadas.	ErrorCode	–
MoveRelative(Double mainDistance, Double subDistance)	Realiza un desplazamiento relativo independiente para cada eje.	ErrorCode	–
MoveAbsoluteAngle(Double mainTargetPosition, Double subAngle)	Desplaza el eje principal a una posición absoluta mientras posiciona el eje secundario según un ángulo especificado.	ErrorCode	–
MoveRelativeAngle(Double mainDistance, Double subAngle)	Realiza un desplazamiento relativo del eje principal y posiciona el eje secundario mediante un ángulo especificado.	ErrorCode	–
Stop()	Detiene el movimiento de ambos ejes.	ErrorCode	–
GetTargetPosition()	Obtiene la posición objetivo del sistema bidimensional.	Point	–
GetActualPosition()	Obtiene la posición actual del sistema bidimensional.	Point	–
IsMoving()	Indica si alguno de los ejes se encuentra en movimiento.	Boolean	–
SetSpeed(Double speed)	Configura la velocidad de desplazamiento del sistema bidimensional.	ErrorCode	–
SetAccelRate(Double rate)	Configura la tasa de aceleración del movimiento.	ErrorCode	–
SetDecelRate(Double rate)	Configura la tasa de desaceleración del movimiento.	ErrorCode	–
GetErrorCode()	Obtiene el código de error actual del sistema bidimensional.	ErrorCode	–
GetStatus()	Obtiene el estado actual del sistema bidimensional.	Status	–
Tipos anidados / Enums			
Point	Estructura que representa una posición bidimensional mediante las coordenadas de los ejes principal y secundario.	Struct	–

2.4 Clase Axis3D

Table 2.4: Miembros de la clase `Axis3D`

Miembro	Descripción	Tipo .NET	Acceso
Constructores			

Continued on next page

Table 2.4: Miembros de la clase `Axis3D` (Continued)

Axis3D(UInt16 mainStageNumberX, UInt16 mainStageNumberY, UInt16 mainStageNumberZ)	Inicializa una nueva instancia de la clase asociada a un sistema de posicionamiento tridimensional compuesto por tres ejes.	Constructor	–
Métodos			
SetAxisNumber(UInt16 mainStageNumberX, UInt16 mainStageNumberY, UInt16 mainStageNumberZ)	Asigna los identificadores de los ejes utilizados por el sistema tridimensional.	ErrorCode	–
GetAxisNumber()	Obtiene los identificadores de los ejes asociados al sistema tridimensional.	UInt16[]	–
MoveAbsolute(Point3D targetPosition)	Desplaza el sistema tridimensional hasta una posición absoluta especificada mediante una estructura de coordenadas.	ErrorCode	–
MoveAbsolute(Double targetPositionX, Double targetPositionY, Double targetPositionZ)	Desplaza el sistema tridimensional hasta las posiciones absolutas indicadas para cada eje.	ErrorCode	–
MoveAbsolute(Point3D targetPosition, UInt16 dOutId, Int16 dOutOnDelayTime, Int16 dOutOffDelayTime)	Desplaza el sistema tridimensional hasta una posición absoluta activando una salida digital durante el movimiento según los tiempos especificados.	ErrorCode	–
MoveAbsolute(Double targetPositionX, Double targetPositionY, Double targetPositionZ, UInt16 dOutId, Int16 dOutOnDelayTime, Int16 dOutOffDelayTime)	Desplaza el sistema tridimensional hasta las posiciones absolutas indicadas para cada eje activando una salida digital durante el movimiento.	ErrorCode	–
MoveRelative(Point3D targetDistance)	Realiza un desplazamiento relativo del sistema tridimensional utilizando una estructura de coordenadas.	ErrorCode	–
MoveRelative(Double targetDistanceX, Double targetDistanceY, Double targetDistanceZ)	Realiza un desplazamiento relativo independiente para cada eje.	ErrorCode	–
MoveRelative(Point3D targetDistance, UInt16 dOutId, Int16 dOutOnDelayTime, Int16 dOutOffDelayTime)	Realiza un desplazamiento relativo activando una salida digital durante el movimiento según los tiempos especificados.	ErrorCode	–
MoveRelative(Double targetDistanceX, Double targetDistanceY, Double targetDistanceZ, UInt16 dOutId, Int16 dOutOnDelayTime, Int16 dOutOffDelayTime)	Realiza un desplazamiento relativo independiente para cada eje activando una salida digital durante el movimiento.	ErrorCode	–
Stop()	Detiene el movimiento de los tres ejes.	ErrorCode	–
GetTargetPosition()	Obtiene la posición objetivo del sistema tridimensional.	Point3D	–
GetActualPosition()	Obtiene la posición actual del sistema tridimensional.	Point3D	–
IsMoving()	Indica si alguno de los ejes se encuentra en movimiento.	Boolean	–

Continued on next page

Table 2.4: Miembros de la clase `Axis3D` (Continued)

SetSpeed(Double speed)	Configura la velocidad de desplazamiento del sistema tridimensional.	ErrorCode	–
SetAccelRate(Double accelRate)	Configura la tasa de aceleración del movimiento.	ErrorCode	–
SetDecelRate(Double decelRate)	Configura la tasa de desaceleración del movimiento.	ErrorCode	–
SetStageNumber(UInt16 mainStageNumberX, UInt16 mainStageNumberY, UInt16 mainStageNumberZ)	Configura los identificadores de los ejes principales utilizados por el sistema.	ErrorCode	–
SetSubStageNumber(UInt16 subStageNumberX, UInt16 subStageNumberY, UInt16 subStageNumberZ)	Configura los identificadores de los ejes secundarios utilizados por el sistema.	ErrorCode	–
SetRotationCenter(Double rotationOffsetX, Double rotationOffsetY, Double rotationOffsetZ)	Define el centro de rotación del sistema mediante los desplazamientos especificados para los ejes X, Y y Z.	ErrorCode	–
GetErrorCode()	Obtiene el código de error actual del sistema tridimensional.	ErrorCode	–
GetStatus()	Obtiene el estado actual del sistema tridimensional.	Status	–
Tipos anidados / Estructuras			
Point3D	Estructura que representa una posición tridimensional mediante las coordenadas X, Y y Z.	Struct	–
RotationCenter	Estructura que define los ejes y desplazamientos utilizados como centro de rotación del sistema.	Struct	–

2.5 Clase Alignment

Table 2.5: Miembros de la clase `Alignment`

Miembro	**Descripción**	**Tipo .NET**	**Acceso**
Constructores			
Alignment()	Inicializa una nueva instancia de la clase `Alignment`.	Constructor	–
Alignment(UInt16 id)	Inicializa una nueva instancia de la clase asociada al identificador de alineación especificado.	Constructor	–
Métodos			
StartSingle()	Inicia el procedimiento de alineación de tipo *Single*.	ErrorCode	–
StartFlat()	Inicia el procedimiento de alineación de tipo *Flat*.	ErrorCode	–
StartFocus()	Inicia el procedimiento de alineación de tipo *Focus*.	ErrorCode	–
StartTwoChRotation()	Inicia el procedimiento de alineación por rotación de dos canales.	ErrorCode	–

Continued on next page

Table 2.5: Miembros de la clase `Alignment` (Continued)

StartMMFRotation()	Inicia el procedimiento de alineación por rotación para fibra multimodo (MMF).	ErrorCode	–
StartLoopBack()	Inicia el procedimiento de alineación de tipo *LoopBack*.	ErrorCode	–
StartCommon()	Inicia un procedimiento de alineación utilizando parámetros comunes.	ErrorCode	–
Stop()	Detiene el procedimiento de alineación en ejecución.	ErrorCode	–
GetErrorCode()	Obtiene el código de error actual del proceso de alineación.	ErrorCode	–
GetStatus()	Obtiene el estado actual del proceso de alineación.	Status	–
GetErrorAxisID()	Obtiene el identificador del eje responsable del error actual.	UInt32	–
GetAligningStatus()	Obtiene el estado detallado del procedimiento de alineación.	StatusCode	–
GetProfilePacketSumIndex (ProfileDataType profileDataType)	Obtiene el número total de paquetes de datos disponibles para el perfil especificado.	UInt32	–
RequestProfileData(ProfileDataType profileDataType, UInt32 index)	Solicita un paquete de datos correspondiente al perfil y al índice especificados.	ProfileData	–
RequestProfileData(ProfileDataType profileDataType, UInt32 index, Boolean isRotationCenterShift)	Solicita un paquete de datos del perfil considerando el desplazamiento del centro de rotación.	ProfileData	–
GetVoltage(UInt32 alignmentAInCh)	Obtiene el valor de tensión medido en el canal de entrada analógica especificado.	Double	–
GetPower(UInt32 alignmentPMCh)	Obtiene la potencia óptica medida por el canal del medidor de potencia especificado.	Double	–
GetMeasurementWaveLength(UInt32 alignmentPMCh)	Obtiene la longitud de onda configurada para el canal del medidor de potencia especificado.	Double	–
SetSingle(ISingleParameter parameter)	Configura los parámetros del procedimiento de alineación *Single*.	ErrorCode	–
SetSingle(ISingleParameter parameter, RotationCenter rotationCenter)	Configura los parámetros del procedimiento *Single* utilizando un centro de rotación definido.	ErrorCode	–
SetFlat(IFlatParameter parameter)	Configura los parámetros del procedimiento de alineación *Flat*.	ErrorCode	–
SetFlat(IFlatParameter parameter, RotationCenter rotationCenter)	Configura los parámetros del procedimiento *Flat* utilizando un centro de rotación definido.	ErrorCode	–
SetFocus(IFocusParameter parameter)	Configura los parámetros del procedimiento de alineación *Focus*.	ErrorCode	–
SetFocus(IFocusParameter parameter, RotationCenter rotationCenter)	Configura los parámetros del procedimiento *Focus* utilizando un centro de rotación definido.	ErrorCode	–
SetTwoChRotation(ITwoChRotationParameter parameter)	Configura los parámetros del procedimiento de alineación por rotación de dos canales.	ErrorCode	–
SetTwoChRotation(ITwoChRotationParameter parameter, RotationCenter rotationCenter)	Configura los parámetros del procedimiento de rotación de dos canales utilizando un centro de rotación definido.	ErrorCode	–

Continued on next page

Table 2.5: Miembros de la clase `Alignment` (Continued)

SetMMFRotation(IMMFRotationParameter parameter)	Configura los parámetros del procedimiento de alineación por rotación para fibra multimodo.	ErrorCode	–
SetMMFRotation(IMMFRotationParameter parameter, RotationCenter rotationCenter)	Configura los parámetros del procedimiento de rotación para fibra multimodo utilizando un centro de rotación definido.	ErrorCode	–
SetLoopBack(ILoopBackParameter parameter)	Configura los parámetros del procedimiento de alineación *LoopBack*.	ErrorCode	–
SetLoopBack(ILoopBackParameter parameter, RotationCenter rotationCenter)	Configura los parámetros del procedimiento *LoopBack* utilizando un centro de rotación definido.	ErrorCode	–
SetCommon(CommonParameter parameter)	Configura los parámetros comunes del procedimiento de alineación.	ErrorCode	–
SetCommon(CommonParameter parameter, RotationCenter rotationCenter)	Configura los parámetros comunes utilizando un centro de rotación definido.	ErrorCode	–
SetMeasurementWaveLength(UInt32 alignmentPMCh, Double waveLength)	Configura la longitud de onda de medición para el canal del medidor de potencia especificado.	ErrorCode	–
Tipos anidados			
AlignmentType	Enumeración que define los distintos tipos de procedimientos de alineación soportados.	Enum	–
ErrorCode	Enumeración que define los posibles códigos de error del proceso de alineación.	Enum	–
Status	Enumeración que representa los estados posibles del procedimiento de alineación.	Enum	–
AligningStatusCode	Enumeración que describe el estado interno del algoritmo de alineación.	Enum	–
ProfileDataType	Enumeración que identifica los diferentes tipos de perfiles generados durante la alineación.	Enum	–
ZMode	Enumeración que define el modo de búsqueda utilizado sobre el eje Z.	Enum	–
ISharingParameter	Interfaz que define los parámetros compartidos entre distintos procedimientos de alineación.	Interface	–
ISingleParameter	Interfaz que define los parámetros del procedimiento *Single*.	Interface	–
SingleParameter	Clase que implementa los parámetros del procedimiento *Single*.	Class	–
IFlatParameter	Interfaz que define los parámetros del procedimiento *Flat*.	Interface	–
FlatParameter	Clase que implementa los parámetros del procedimiento *Flat*.	Class	–
IFocusParameter	Interfaz que define los parámetros del procedimiento *Focus*.	Interface	–
FocusParameter	Clase que implementa los parámetros del procedimiento *Focus*.	Class	–
ITwoChRotationParameter	Interfaz que define los parámetros del procedimiento de rotación de dos canales.	Interface	–

Continued on next page

Table 2.5: Miembros de la clase `Alignment` (Continued)

TwoChRotationParameter	Clase que implementa los parámetros del procedimiento de rotación de dos canales.	Class	–
IMMFRotationParameter	Interfaz que define los parámetros del procedimiento de rotación para fibra multimodo.	Interface	–
MMFRotationParameter	Clase que implementa los parámetros del procedimiento de rotación para fibra multimodo.	Class	–
ILoopBackParameter	Interfaz que define los parámetros del procedimiento *LoopBack*.	Interface	–
LoopBackParameter	Clase que implementa los parámetros del procedimiento *LoopBack*.	Class	–
ICommonParameter	Interfaz que define los parámetros comunes de los procedimientos de alineación.	Interface	–
CommonParameter	Clase que implementa los parámetros comunes de alineación.	Class	–
ProfileData	Clase que almacena los datos de perfil obtenidos durante un procedimiento de alineación.	Class	–

2.6 Clase Profile

Table 2.6: Miembros de la clase `Profile`

Miembro	**Descripción**	**Tipo .NET**	**Acceso**
Constructores			
Profile()	Inicializa una nueva instancia de la clase `Profile`.	Constructor	–
Métodos			
Start()	Inicia el proceso de adquisición de datos de perfil.	ErrorCode	–
Stop()	Detiene el proceso de adquisición de datos de perfil.	ErrorCode	–
GetProfileStatus()	Obtiene el estado actual del proceso de adquisición del perfil.	Status	–
GetProfilePacketSumIndex()	Obtiene el número total de paquetes de datos disponibles para el perfil adquirido.	UInt32	–
RequestProfileData(UInt32 index)	Solicita un paquete de datos de perfil correspondiente al índice especificado.	ProfileData	–
RequestProfileData(UInt32 index, Boolean isRotationCenterShift)	Solicita un paquete de datos de perfil considerando el desplazamiento del centro de rotación.	ProfileData	–
SetProfile(ProfileParameter parameter)	Configura los parámetros utilizados para la adquisición del perfil.	ErrorCode	–
SetProfile(ProfileParameter parameter, RotationCenter rotationCenter)	Configura los parámetros de adquisición del perfil utilizando un centro de rotación definido.	ErrorCode	–
Tipos anidados			
ErrorCode	Enumeración que define los posibles códigos de error del proceso de adquisición del perfil.	Enum	–

Continued on next page

Table 2.6: Miembros de la clase `Profile` (Continued)

Status	Enumeración que representa los estados posibles del proceso de adquisición del perfil.	Enum	–
ProfileParameter	Clase que define los parámetros utilizados para configurar la adquisición de un perfil.	Class	–
ProfileData	Clase que almacena los datos obtenidos durante la adquisición de un perfil.	Class	–

2.7 Clase Angle Asjustment

Table 2.7: Miembros de la clase `AngleAdjustment`

Miembro	Descripción	Tipo .NET	Acceso
Constructores			
AngleAdjustment(UInt16 id)	Inicializa una nueva instancia de la clase asociada al identificador del módulo de ajuste angular especificado.	Constructor	–
Métodos			
Start()	Inicia el procedimiento de ajuste angular.	ErrorCode	–
Stop()	Detiene el procedimiento de ajuste angular.	ErrorCode	–
GetStatus()	Obtiene el estado actual del procedimiento de ajuste angular.	Status	–
GetErrorAxisID()	Obtiene el identificador del eje responsable del error actual.	UInt32	–
GetAdjustingStatus()	Obtiene el estado detallado del procedimiento de ajuste angular.	AdjustingStatus	–
GetProfilePacketSumIndex(ProfileDataType profileDataType)	Obtiene el número total de paquetes de datos disponibles para el tipo de perfil especificado.	UInt32	–
RequestProfileData(ProfileDataType profileDataType, UInt32 index)	Solicita un paquete de datos correspondiente al perfil y al índice especificados.	ProfileData	–
SetParameter(AngleAdjustmentParameter parameter)	Configura los parámetros utilizados durante el procedimiento de ajuste angular.	ErrorCode	–
SetParameter(AngleAdjustmentParameter parameter, RotationCenter rotationCenter)	Configura los parámetros del procedimiento utilizando un centro de rotación definido.	ErrorCode	–
Tipos anidados			
ErrorCode	Enumeración que define los posibles códigos de error del procedimiento de ajuste angular.	Enum	–
Status	Enumeración que representa los estados posibles del procedimiento de ajuste angular.	Enum	–
AdjustingStatus	Enumeración que describe el estado interno del algoritmo de ajuste angular.	Enum	–
ProfileDataType	Enumeración que identifica los diferentes tipos de perfiles generados durante el procedimiento de ajuste angular.	Enum	–
AngleAdjustmentParameter	Clase que define los parámetros utilizados para configurar el procedimiento de ajuste angular.	Class	–

Continued on next page

Table 2.7: Miembros de la clase `AngleAdjustment` (Continued)

ProfileData	Clase que almacena los datos de perfil obtenidos durante el procedimiento de ajuste angular.	Class	–

Chapter 3

Routines

3.1 RasterScan

The routine performs a two-dimensional raster scan by moving along a selected axis while acquiring a profile along the perpendicular axis at each scan position. For example, when the X axis is selected as the scan axis, the system moves along X according to the specified scan step and acquires a Z-profile at each X position. This approach is particularly useful for vertical coupling measurements, where the optical power distribution can be characterized as a function of the X and Z positions.

During the scan, the profiles acquired using the Suruga Seiki utilities can be saved for further analysis. The acquired data are then cleaned by removing invalid or irrelevant samples that could interfere with the generation of the heatmap. The processed profiles are interpolated onto a common grid and used to generate an interactive heatmap representing the measured power distribution.

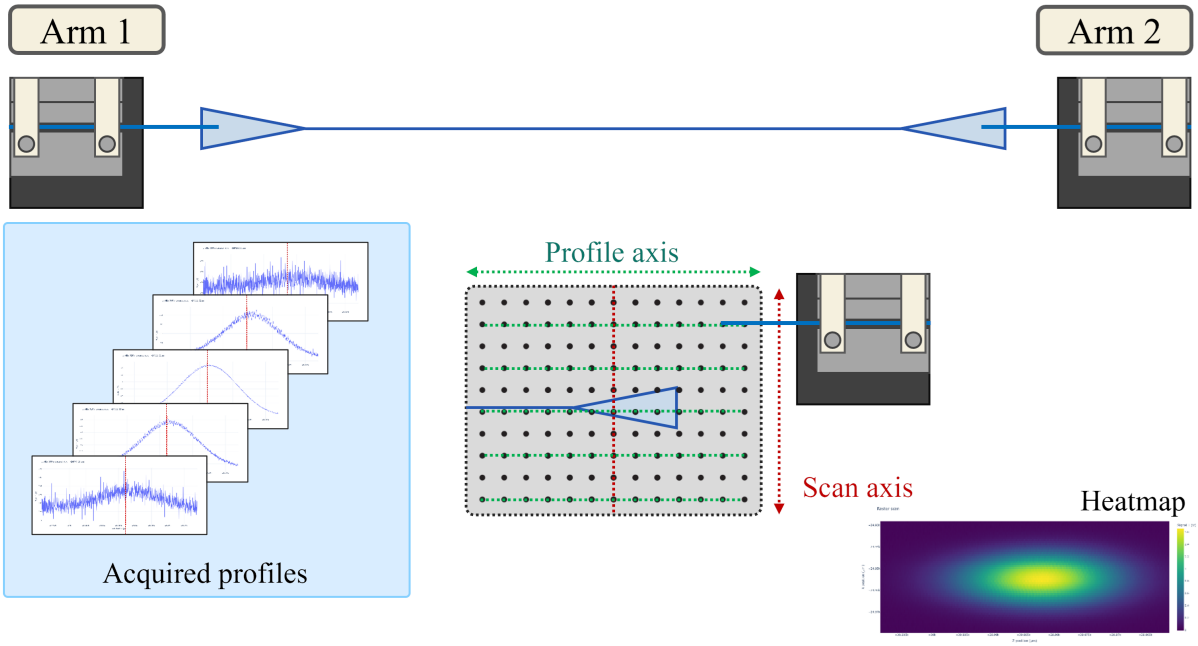


Figure 3.1: Two-dimensional raster scan for optical coupling localization. Profiles are acquired along the profile axis while the stage is incrementally moved along the perpendicular scan axis. The acquired profiles are combined to reconstruct a two-dimensional heatmap, allowing the position of maximum optical coupling to be identified.

Once the scan is complete, the routine identifies the global maximum power measured within the scanned area and, if enabled, automatically moves the arm to the corresponding X and Z position.

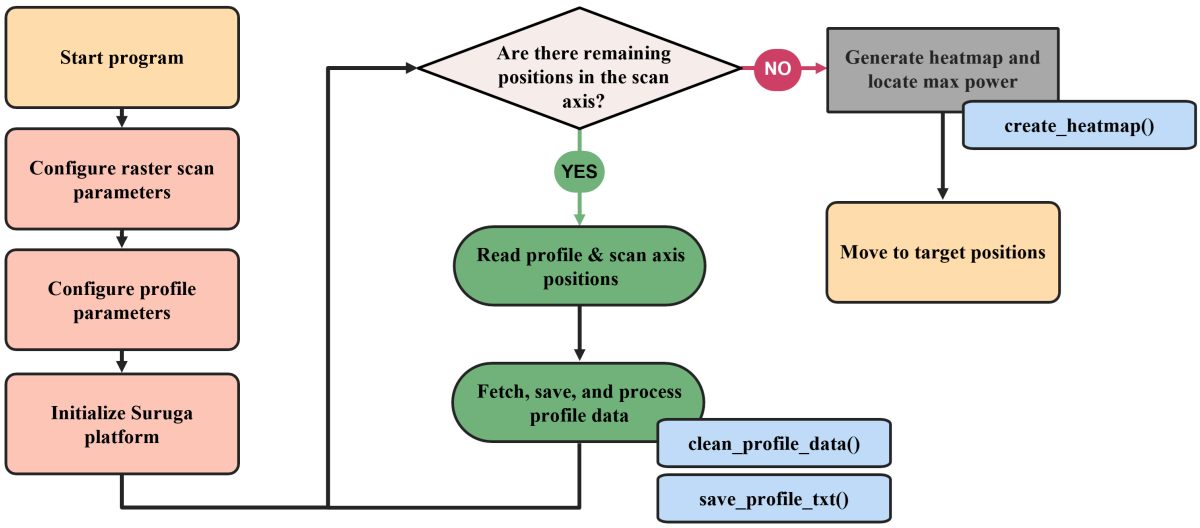


Figure 3.2: Flowchart of the *RasterScan* routine. The process begins with the configuration of the raster scan and profile parameters, followed by the initialization of the Suruga Seiki platform. For each position along the scan axis, the profile and scan-axis positions are obtained, and the acquired data are retrieved, stored, and processed using `clean_profile_data()` and `save_profile_txt()`. The process is repeated until all positions along the scan axis have been completed. Finally, the processed profiles are passed to `create_heatmap()` to reconstruct the two-dimensional power map and locate the maximum signal, after which the system can move to the corresponding target positions.

3.1.1 `clean_profile_data()`

Objective:

Clean and prepare the data acquired by the profile function for further processing.

This function receives the complete dictionary returned by `profile.get_profile_data()` and extracts only the variables required for the raster scan:

- `main_position`: the position of the axis being profiled.
- `signal1`: the measured optical signal.

The function then performs the following operations:

1. Converts the extracted data into NumPy arrays.
2. Removes invalid samples, including:
    - Non-finite position values.
    - Non-finite signal values.
    - Samples where the position is equal to zero.
3. Sorts and reorganizes the remaining samples according to their position.

The function **does not modify or save the original profile data**. Its purpose is to generate a cleaner and more manageable representation of the acquired data, which can then be used for further processing, particularly for generating the heatmap with `create_heatmap()`.

3.1.2 `save_profile_txt()`

Objective:

Preserve the complete profile data acquired during the measurement, together with the experimental parameters required to document and reproduce the measurement.

This function receives the complete dictionary returned by `profile.get_profile_data()`, including the following variables:

- `main_position`: position of the main axis during the profile.
- `sub1_position`: position of the first secondary axis.
- `sub2_position`: position of the second secondary axis.
- `signal1`: first measured signal.
- `signal2`: second measured signal.

In addition to the acquired data, the function constructs a header containing the experimental parameters and the stage positions at the time of acquisition:

- **Date:** Date and time of the measurement.
- **Center X:** X-axis center position.
- **Center Z:** Z-axis center position.
- **Arm:** Selected measurement arm or configuration.
- **Scan axis:** Axis along which the raster scan is performed.
- **Scan width:** Total width of the scan.
- **Scan step:** Distance between consecutive scan positions.
- **Profile axis:** Axis along which the profile is acquired.
- **Profile range:** Total range covered by the profile.
- **Profile speed:** Speed used during profile acquisition.

The function then stores the complete dataset and the associated metadata in a **tab-separated TXT file** using NumPy's `np.savetxt()` function.

Unlike `clean_profile_data()`, this function does **not** remove, filter, or reorganize the acquired samples. Its purpose is to preserve the original measurement data as completely as possible, while providing the experimental metadata necessary to identify and reproduce the measurement.

The resulting TXT file therefore contains both the **raw profile data** and the **measurement parameters** associated with its acquisition.

3.1.3 `create_heatmap()`

Objective:

Reconstruct a two-dimensional signal map from all the profiles acquired during the raster scan, generate an interactive heatmap, and determine the global maximum of `signal1` together with its corresponding X and Z coordinates.

The function receives two inputs:

- `scan_data`: a dictionary containing the processed profiles acquired during the raster scan. Each entry has the following structure:

```
[
    {
        "x": ...,
        "z_positions": ...,
        "signal": ...
    },
    ...
]
```

- `output_path`: the path where the generated heatmap will be saved.

Data Reconstruction

First, the function takes the Z positions from the first acquired profile and uses them as the common reference axis:

```
z_common = scan_data[0]["z_positions"]
```

Since the Z positions may not be identical for every profile, each profile is interpolated onto this common Z-axis. This ensures that all profiles have the same number of samples and can be represented consistently in a two-dimensional matrix.

For each profile, the function:

1. Retrieves the corresponding X position.
2. Retrieves the Z positions and measured signal.
3. Interpolates the signal onto `z_common`.
4. Adds the interpolated signal to the 2D signal matrix.

The resulting matrix represents the measured signal as a function of the X and Z coordinates and is used to generate the heatmap.

Heatmap Generation

The reconstructed 2D signal matrix is used to generate an **interactive Plotly heatmap**, where:

- The **X-axis** represents the raster scan position.
- The **Z-axis** represents the profile position.
- The **color scale** represents the measured `signal1`.

The resulting heatmap provides a two-dimensional visualization of the measured optical signal across the scanned area and is saved to the location specified by `output_path`.

Global Maximum Detection

In addition to generating the heatmap, the function determines the position of the maximum measured signal.

For each profile, the function first identifies its **local maximum**. The local maximum is then compared with the maximum found in the previous profiles. By continuously retaining the highest value, the function ultimately determines the **global maximum** across the entire raster scan.

The global maximum is stored together with its corresponding coordinates:

- **Maximum signal:** maximum value of `signal1`.
- **X coordinate:** X position of the profile where the maximum was found.
- **Z coordinate:** Z position corresponding to the maximum signal within that profile.

Therefore, `create_heatmap()` performs two main tasks: it **reconstructs and visualizes the complete raster scan as a two-dimensional signal map**, and it **identifies the position of the maximum output power** within the scanned region.

3.2 Automaper

Source code: automaper.py.

`automaper.py` provides a set of utilities for loading, interpreting, visualizing, and managing photonic chip layouts described through an Excel-based chip map.

The module acts as an intermediate layer between the logical description of a photonic chip and the measurement software responsible for its physical characterization. Instead of hardcoding device coordinates, port locations, or measurement combinations in Python, the chip structure is defined externally in an Excel workbook. Automaper parses this information and converts it into structured Python data models that can be used by the positioning and measurement routines.

The general workflow is:

```
Excel chip map
      ↓
AUTOMAPER
      ↓
Chip data model
      ↓
Measurement paths
      ↓
Measurement plan
      ↓
Positioning / measurement software
```

The main purpose of automaper.py is therefore measurement-plan generation and management, rather than direct hardware control. The module does not move the positioning stages, acquire optical power, or execute the measurements itself. Instead, it produces the structured information required by other parts of the characterization software to perform these operations.

3.2.1 Building blocks

Automaper can be divided into three functional blocks:

1. **Chip representation:** Loads the device and port definitions from the Excel workbook and converts them into the internal `chip` data structure.
2. **Measurement-path management:** Defines the possible input-to-output connections and allows the user to enable or disable individual measurement paths through the `MeasurementPaths` worksheet.
3. **Measurement-plan generation:** Combines the chip topology with the enabled measurement paths and optional coupling constraints to generate the final `measurement_plan`.

The principal data structures generated by the module are therefore:

- chip
- measurement_paths

- measurement_plan

These structures represent progressively more specific levels of information, from the physical description of the chip to the exact measurements that should be executed.

Table 3.1: Worksheets used to define the chip measurement configuration.

Worksheet	Description
Metadata	Defines the context of the measurement.
Devices	Defines the devices available for measurement.
Ports	Defines the ports associated with each device, including their position, direction, and coupling type.
MeasurementPaths	Defines the input-to-output measurement paths to be considered.

3.2.2 checkPositionsAutomap

Validates the measurement plan generated by AutoMapper before moving the Suruga platform. It calculates and displays the absolute `x1` and `x2` positions for each device defined in the Excel file, but it does not connect to any instrument or move any axes.

Source code: `checkPositionsAutomap.py`.

Workflow

1. Loads the chip layout and measurement plan from an Excel file using `automaper.load_chip()`.
2. Uses `origin_x1` and `origin_x2` as the reference origins.
3. For each measurement, calculates the input and output offsets and the absolute positions of both arms.
4. Displays the device, ports, and calculated positions, and waits for confirmation before displaying the next measurement.

Relevant Configuration

Table 3.2: Relevant configuration parameters for `checkPositionsAutomap`.

Parameter	Initial Value	Description
`EXCEL_FILE`	`chipMaps/SurugaTest.xlsx`	Chip layout and AutoMapper measurement plan.
`origin_x1 / origin_x2`	0.0 μm / 0.0 μm	Reference positions used to calculate the absolute coordinates.

The absolute coordinates are calculated according to

$$x_1 = \text{origin}_{x1} - \text{input}.x \tag{3.1}$$

and

$$x_2 = \text{origin}_{x2} - \text{output}.x. \tag{3.2}$$

Recommended Usage

Run this routine whenever the Excel file, chip coupling, or origin convention changes. Compare the printed positions against the mechanical setup limits before running a routine that moves the machine.

Results

This routine does not create files or perform measurements. Its output consists of the position list displayed in the console.

3.2.3 moveAutomapOPM

Automatically iterates through the measurements defined by AutoMapper, positions the fibers at the input and output waveguides, performs alignment on both arms, and measures the average optical power using the optical power meter (OPM).

Source code: `moveAutomapOPM.py`.

3.2.4 Workflow

1. Loads the measurement plan from the AutoMapper Excel file.
2. Connects to the Suruga platform and creates controllers for `x1`, `x2`, alignment, and the power meter.
3. Loads `origin.txt` or uses the current position as the `WG0` origin.
4. For each device, calculates the absolute positions of both arms and moves `x1` and `x2`.
5. Performs a *flat* alignment on the left arm followed by another alignment on the right arm.
6. Reads ten samples from the OPM, calculates their average, and displays the result in the console.
7. Requests confirmation before continuing and, once all measurements are completed, returns to `WG0`.

3.2.5 Relevant Configuration

Table 3.3: Relevant configuration parameters for `moveAutomapOPM`.

Parameter	Initial Value	Description
`EXCEL_FILE`	`chipMaps/SurugaTest.xlsx`	Chip layout and measurement plan.
`USE_SAVED_ORIGIN / SAVE_ORIGIN`	`True / False`	Control whether `origin.txt` is loaded or saved.
`ORIGIN_FILE`	`origin.txt`	File containing the X1 and X2 coordinates of `WG0`.
`flat_left / flat_right`	`---`	Alignment parameters for each arm.

3.2.6 Results

The average optical power for each device is printed to the console. The routine does not save the measurement values to a file.

3.2.7 Precautions

The routine moves both arms and requires manual confirmation between measurements. Verify the position plan using `checkPositionsAutomap` before running it.

3.2.8 moveAutomapOPM_OSA

Characterizes the devices defined by AutoMapper by combining Suruga platform movement, automatic alignment, an average optical power measurement using the OPM, and spectrum acquisition with the OSA.

Source code: `moveAutomapOPM_OSA.py`.

3.2.9 Workflow

1. Loads the chip layout and measurement plan from Excel.
2. Connects to the Suruga platform, OPM, and OSA, and configures the spectral sweep.
3. Loads or saves the `WG0` origin in `origin2.txt`.
4. For each device and repetition, positions `x1` and `x2` based on the corresponding input and output waveguides.
5. Aligns the left and right arms.
6. Runs an OSA sweep, retrieves the trace, and averages ten power readings.
7. Saves the trace along with the device, ports, optical power, and repetition number as metadata.
8. Returns to `WG0` and closes the OSA connection.

3.2.10 Relevant Configuration

Table 3.4: Relevant configuration parameters for `moveAutomapOPM_OSA`.

Parameter	Initial Value	Description
`DEBUG_MODE`	`True`	Requests manual confirmations; when set to `False`, the routine runs automatically.
`USE_SAVED_ORIGIN / SAVE_ORIGIN`	`False / True`	Control whether the origin stored in `origin2.txt` is loaded or saved.
`EXCEL_FILE`	`chipMaps/SurugaTest01.xlsx`	Chip layout and AutoMapper measurement plan.
`num_repeticiones`	`1`	Number of complete passes through the measurement plan.

Continued on next page

Table 3.4: Relevant configuration parameters for `moveAutomapOPM_OSA`. (Continued)

Parameter	Initial Value	Description
`CENTER / SPAN`	`1550 nm / 90 nm`	OSA spectral window.
`RESOLUTION / SENSITIVITY`	`0.2 nm / -65 dBm`	OSA resolution and sensitivity.
`channel`	`1`	Power meter channel.

3.2.11 Results

For each device, the routine saves a trace to:

`data/<date>/medicion_<repetition>_<device>_<input>_<output>.txt`

The file contains the spectral data and metadata including the device, ports, average optical power, and repetition number.

3.2.12 Precautions

With `DEBUG_MODE=True`, the routine pauses before the OSA sweep, between devices, and before returning to the origin.

Before using automatic mode, validate the movements with `checkPositionsAutomap`, verify the origin, and confirm that the OSA VISA resource is available.

Appendix A

Automap

Automaper is a python script that converts a simple Excel description of a photonic chip into a complete measurement plan ready for chip characterization.

Instead of hardcoding device coordinates or manually programming measurement routines, the user only needs to describe the chip structure in an Excel file then automaper.py interprets this information and automatically generates the data structures required by the positioning software.

The Excel file is organized in independent sheets, where each sheet describes a different aspect of the chip as follows:

Metadata

Contains general information about the chip, answers the question: What is the context of this measurement?.

Typical information includes

- Project
- Wafer
- Die
- Operator
- Date
- Comments

This information is descriptive only and is not involved in the positioning process.

Devices

This worksheet answers the question: What devices are available on the chip for measurement?

Each row defines a single device together with the information required to identify it throughout the measurement plan.

The columns have the following purposes:

- **Device:** A unique identifier for each device. This identifier is used throughout the workbook to associate the corresponding ports and measurement paths with the correct device.

- **Type:** Specifies the type of photonic device (e.g., waveguide, MMI, ring resonator, splitter). Although this information is not required for positioning, it is useful for filtering devices, generating reports, and applying device-specific measurement or analysis routines.
- **Description:** An optional user-defined description. This field does not affect Automaper's operation or the alignment process; it is provided solely for documentation and traceability purposes.
- **Measure:** Enables or disables a device for the current measurement routine. When unchecked, the device is excluded from the measurement routine without requiring any modification to the measurement paths.

Ports

Where is every optical port located?

Measurement plan

Which optical paths should be measured?

Appendix B

Instalación TwinCAT XAE eXtended Automation Engineering

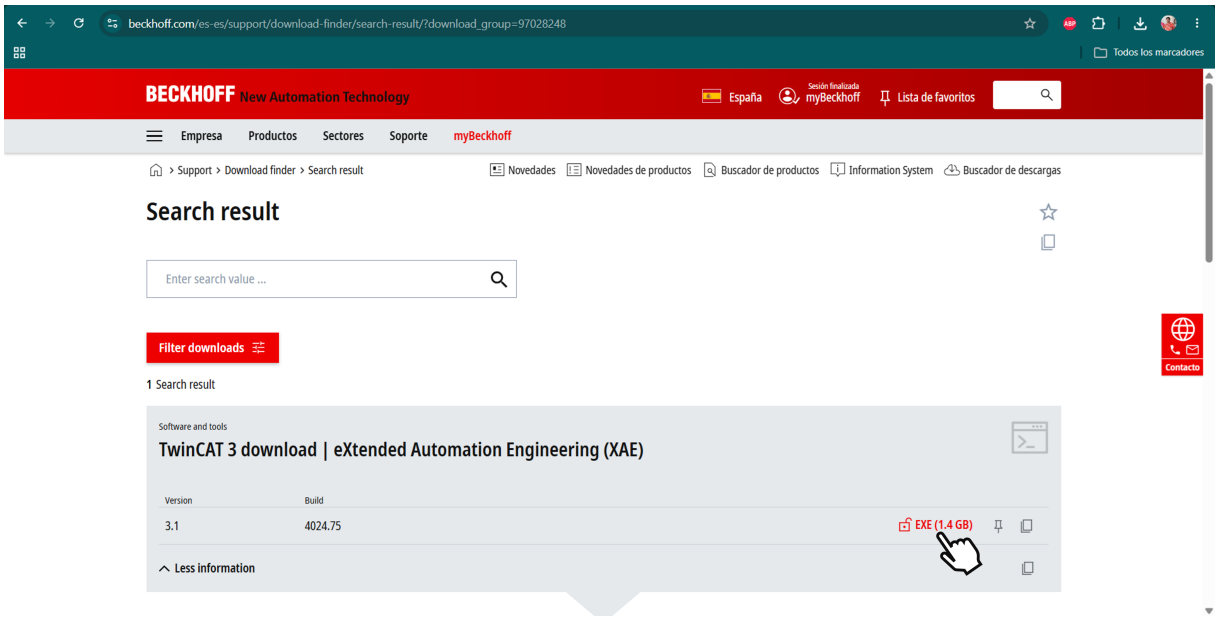


Figure B.1: Pendiente.

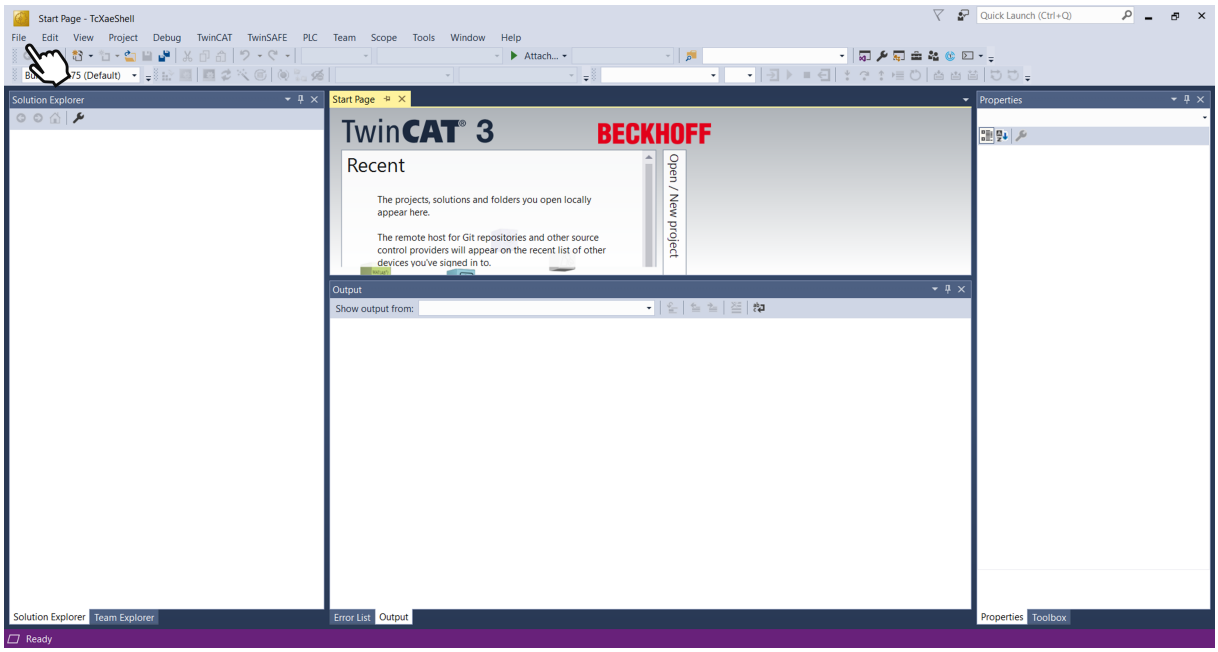


Figure B.2: Pendiente.

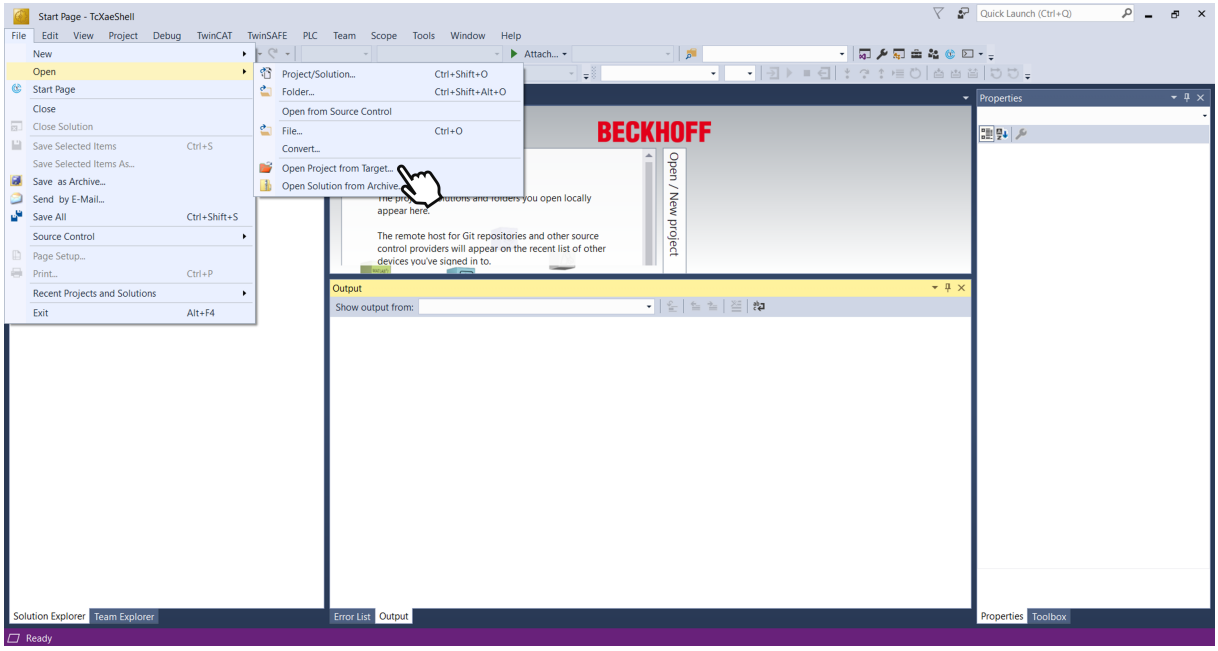


Figure B.3: Pendiente.

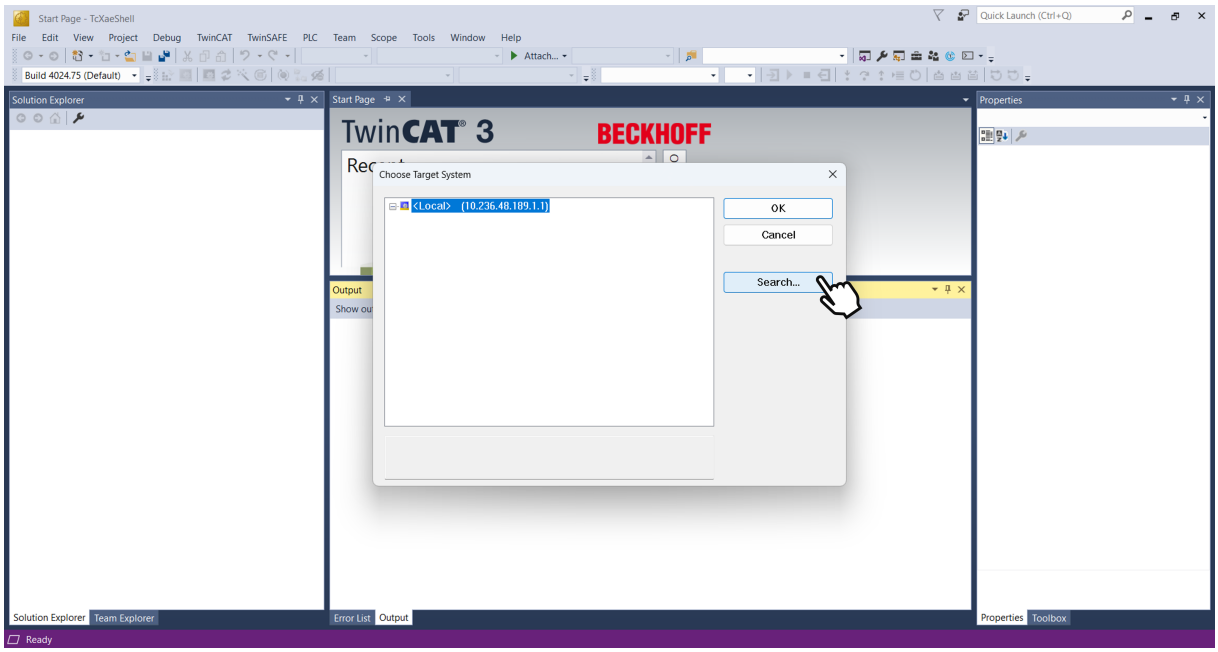


Figure B.4: Pendiente.

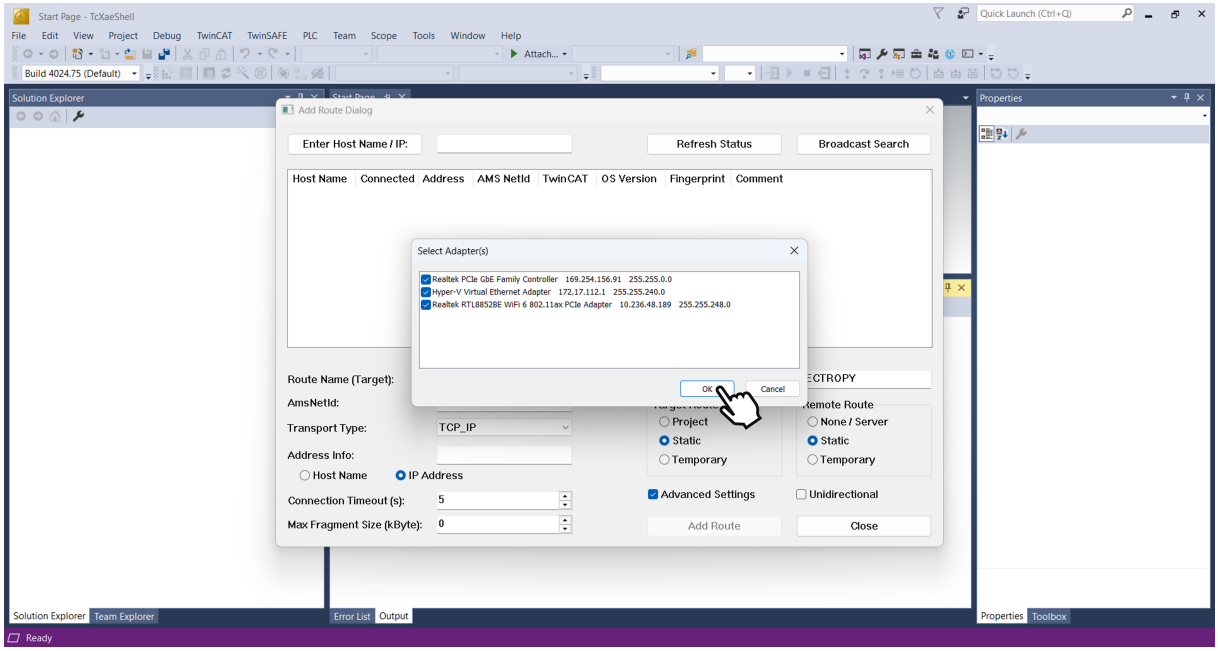


Figure B.5: Pendiente.

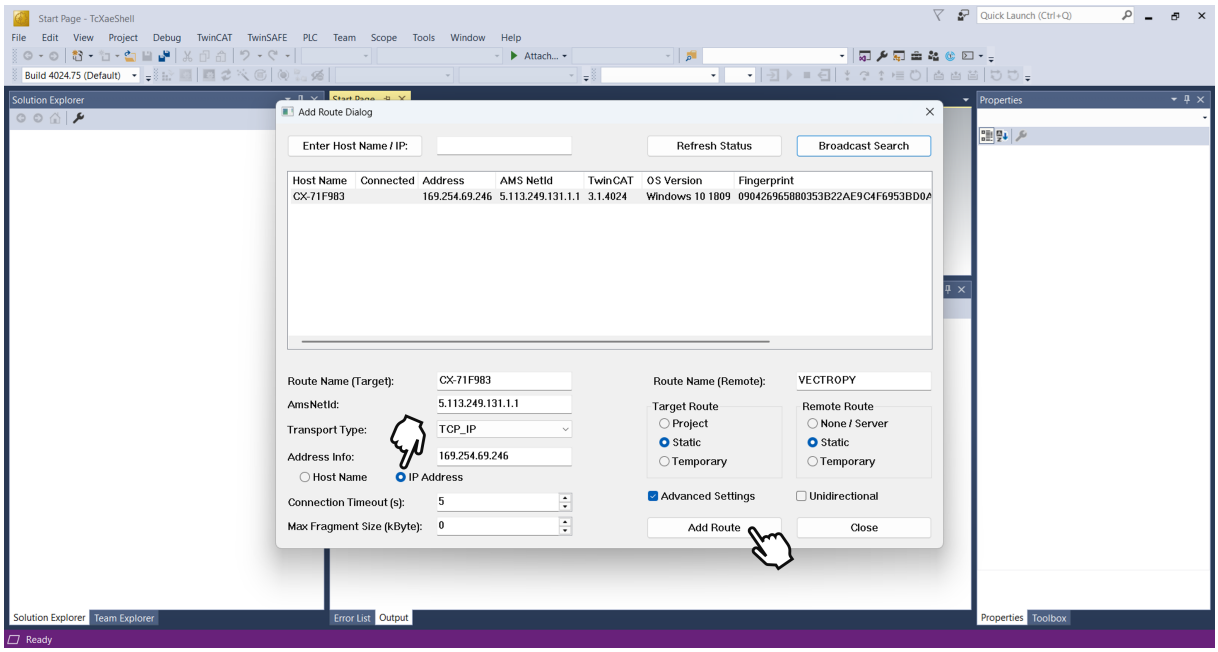


Figure B.6: Pendiente.

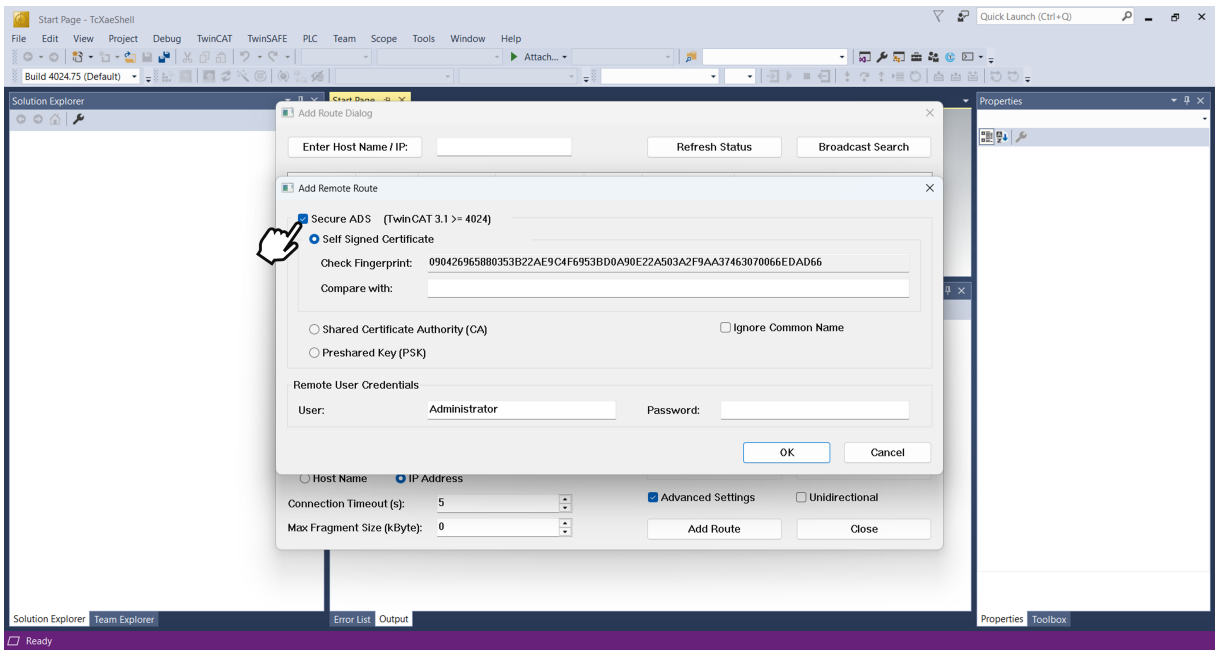


Figure B.7: Pendiente.

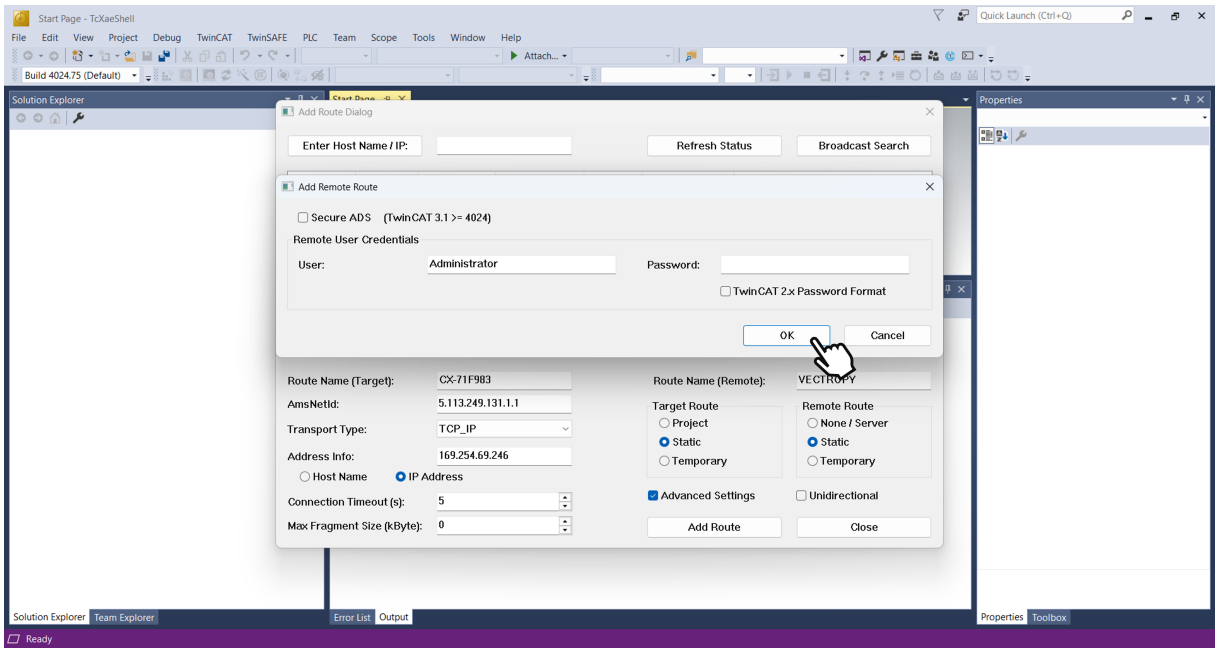


Figure B.8: Pendiente.

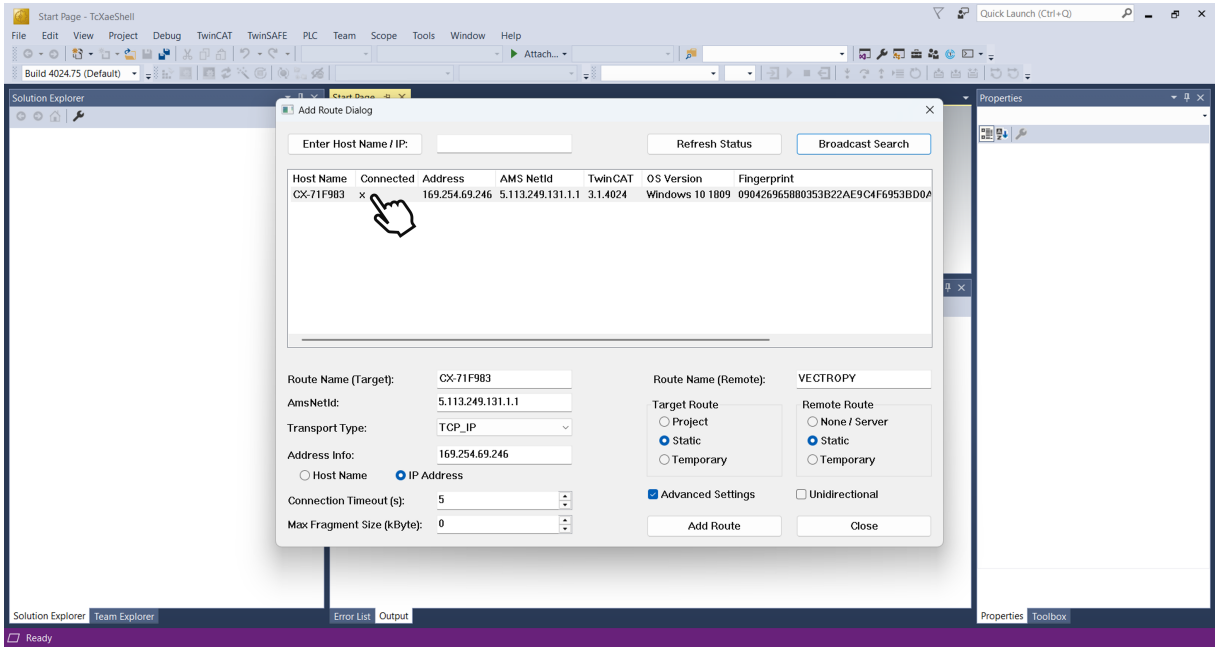


Figure B.9: Pendiente.

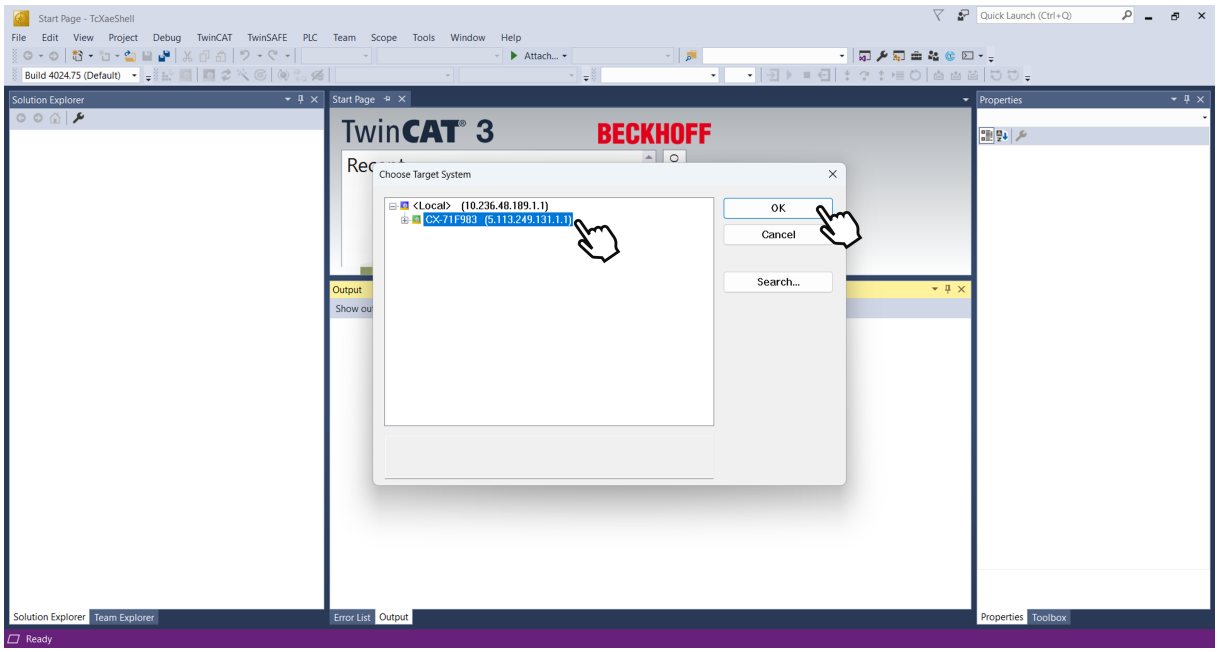


Figure B.10: Pendiente.

Appendix C

Documentacion de la API Suruga Seiki

Es necesario cargar el ensamblado (assembly), una unidad de código previamente compilada que permite establecer la comunicación con la plataforma Suruga Seiki. Este ensamblado está compuesto por un conjunto de clases que definen las propiedades (atributos) y los métodos (acciones o funcionalidades) disponibles en la plataforma.

En programación orientada a objetos, los métodos representan las acciones que una clase puede ejecutar, mientras que las propiedades describen las características asociadas a sus objetos. Un objeto es una instancia concreta de una clase; utilizando una analogía, la clase puede entenderse como un plano de construcción y el objeto como el resultado material obtenido a partir de ese plano.

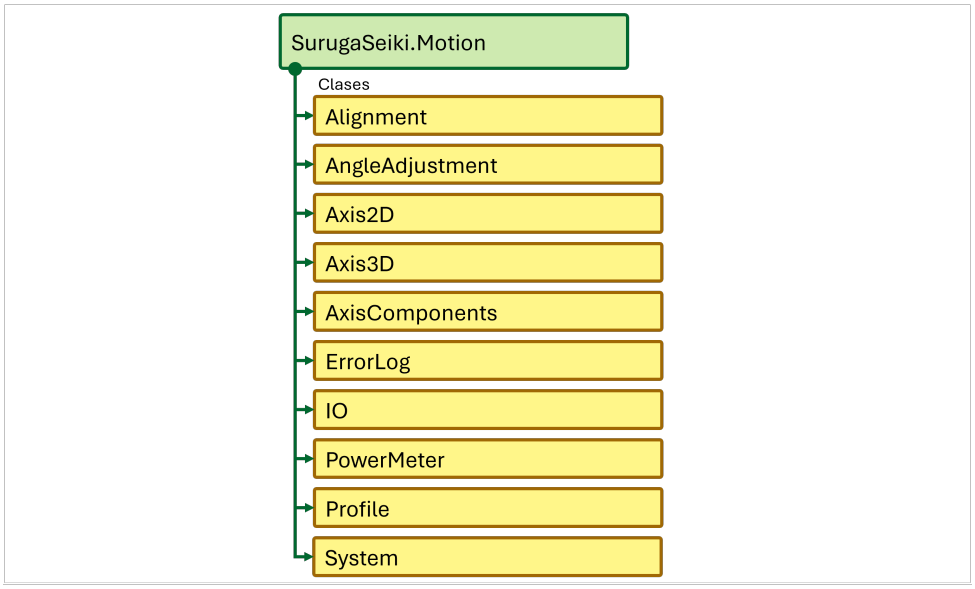


Figure C.1: Pendiente.

C.1 Clase Alignment

```
=============================================================================================
SurugaSeiki.Motion.Alignment
=============================================================================================

Constructores:
```

```
  Alignment()
  Alignment(UInt16 id)

Metodos:
  ErrorCode StartSingle()
  ErrorCode StartFlat()
  ErrorCode StartFocus()
  ErrorCode StartTwoChRotation()
  ErrorCode StartMMFRotation()
  ErrorCode StartLoopBack()
  ErrorCode StartCommon()
  ErrorCode Stop()
  ErrorCode GetErrorCode()
  Status GetStatus()
  UInt32 GetErrorAxisID()
  AligningStatusCode GetAligningStatus()
  UInt32 GetProfilePacketSumIndex(ProfileDataType profileDataType)
  ProfileData RequestProfileData(ProfileDataType profileDataType, UInt32 index)
  ProfileData RequestProfileData(ProfileDataType profileDataType, UInt32 index, Boolean isRotationCenterShift)
  Double GetVoltage(UInt32 alignmentAInCh)
  Double GetPower(UInt32 alignmentPMCh)
  Double GetMeasurementWaveLength(UInt32 alignmentPMCh)
  ErrorCode SetSingle(ISingleParameter parameter)
  ErrorCode SetSingle(ISingleParameter parameter, RotationCenter rotationCenter)
  ErrorCode SetFlat(IFlatParameter parameter)
  ErrorCode SetFlat(IFlatParameter parameter, RotationCenter rotationCenter)
  ErrorCode SetFocus(IFocusParameter parameter)
  ErrorCode SetFocus(IFocusParameter parameter, RotationCenter rotationCenter)
  ErrorCode SetTwoChRotation(ITwoChRotationParameter parameter)
  ErrorCode SetTwoChRotation(ITwoChRotationParameter parameter, RotationCenter rotationCenter)
  ErrorCode SetMMFRotation(IMMFRotationParameter parameter)
  ErrorCode SetMMFRotation(IMMFRotationParameter parameter, RotationCenter rotationCenter)
  ErrorCode SetLoopBack(ILoopBackParameter parameter)
  ErrorCode SetLoopBack(ILoopBackParameter parameter, RotationCenter rotationCenter)
  ErrorCode SetCommon(CommonParameter parameter)
  ErrorCode SetCommon(CommonParameter parameter, RotationCenter rotationCenter)
  ErrorCode SetMeasurementWaveLength(UInt32 alignmentPMCh, Double waveLength)

Tipos anidados/enums:
  SurugaSeiki.Motion.Alignment+AlignmentType
  SurugaSeiki.Motion.Alignment+ErrorCode
  SurugaSeiki.Motion.Alignment+Status
  SurugaSeiki.Motion.Alignment+AligningStatusCode
  SurugaSeiki.Motion.Alignment+ProfileDataType
  SurugaSeiki.Motion.Alignment+ZMode
  SurugaSeiki.Motion.Alignment+ISharingParameter
  SurugaSeiki.Motion.Alignment+ISingleParameter
  SurugaSeiki.Motion.Alignment+SingleParameter
  SurugaSeiki.Motion.Alignment+IFlatParameter
  SurugaSeiki.Motion.Alignment+FlatParameter
  SurugaSeiki.Motion.Alignment+IFocusParameter
  SurugaSeiki.Motion.Alignment+FocusParameter
  SurugaSeiki.Motion.Alignment+ITwoChRotationParameter
  SurugaSeiki.Motion.Alignment+TwoChRotationParameter
  SurugaSeiki.Motion.Alignment+IMMFRotationParameter
  SurugaSeiki.Motion.Alignment+MMFRotationParameter
  SurugaSeiki.Motion.Alignment+ILoopBackParameter
  SurugaSeiki.Motion.Alignment+LoopBackParameter
  SurugaSeiki.Motion.Alignment+ICommonParameter
  SurugaSeiki.Motion.Alignment+CommonParameter
  SurugaSeiki.Motion.Alignment+ProfileData

================================================================================================
SurugaSeiki.Motion.Alignment+AligningStatusCode
================================================================================================

Campos:
  Int32 value__
  static AligningStatusCode NotAligning
  static AligningStatusCode Initializing
  static AligningStatusCode FieldSearching
  static AligningStatusCode PeakSearchingX
  static AligningStatusCode PeakSearchingY
  static AligningStatusCode PeakSearchingZ
  static AligningStatusCode PeakSearchXCh2
  static AligningStatusCode PeakSearchYCh2

================================================================================================
SurugaSeiki.Motion.Alignment+AlignmentType
================================================================================================

Campos:
  Int32 value__
  static AlignmentType Single
  static AlignmentType Flat
  static AlignmentType Focus
  static AlignmentType TwoChRotation
  static AlignmentType MMFRotation
  static AlignmentType LoopBack
  static AlignmentType TwoChFlat

================================================================================================
SurugaSeiki.Motion.Alignment+CommonParameter
================================================================================================

Propiedades:
  AlignmentType alignmentType (get/set)
  ZMode zMode (get/set)
  UInt16 mainStageNumberX (get/set)
```

```
  UInt16 mainStageNumberY (get/set)
  UInt16 mainStageNumberZ (get/set)
  UInt16 subStageNumberX (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleX (get/set)
  Double subAngleY (get/set)
  UInt16 rotateStageNumber (get/set)
  Double rotateSpeed (get/set)
  Int16 rotateDirection (get/set)
  Double coreDistance (get/set)
  UInt16 mmfStageNumber (get/set)
  Double mmfSpeed (get/set)
  Double mmfDistance (get/set)
  UInt16 pmCh (get/set)
  UInt16 pmCh1 (get/set)
  UInt16 pmCh2 (get/set)
  UInt16 analogCh (get/set)
  UInt16 analogCh1 (get/set)
  UInt16 analogCh2 (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Int16 pmInitRange (get/set)
  Int16 pmInitRangeCh1 (get/set)
  Int16 pmInitRangeCh2 (get/set)
  Double fieldSearchThreshold (get/set)
  Double fieldSearchThresholdCh1 (get/set)
  Double fieldSearchThresholdCh2 (get/set)
  Double peakSearchThreshold (get/set)
  Double peakSearchThresholdXY (get/set)
  Double peakSearchThresholdCh1 (get/set)
  Double peakSearchThresholdCh2 (get/set)
  Double peakSearchThresholdZ (get/set)
  Double searchRangeX (get/set)
  Double searchRangeY (get/set)
  Double searchRangeZ (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchPitchZ (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double fieldSearchSpeedZ (get/set)
  Double peakSearchSpeedX (get/set)
  Double peakSearchSpeedY (get/set)
  Double peakSearchSpeedZ (get/set)
  UInt32 smoothingRangeX (get/set)
  UInt32 smoothingRangeY (get/set)
  UInt32 smoothingRangeZ (get/set)
  Double centroidThresholdX (get/set)
  Double centroidThresholdY (get/set)
  Double centroidThresholdZ (get/set)
  Double convergentRangeX (get/set)
  Double convergentRangeY (get/set)
  Double convergentRangeZ (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ErrorCode
================================================================================================

Campos:
  Int32 value__
  static ErrorCode None
  static ErrorCode Axis
  static ErrorCode Aligning
  static ErrorCode Parameter

================================================================================================
SurugaSeiki.Motion.Alignment+FlatParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberX (get/set)
  UInt16 mainStageNumberY (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleX (get/set)
  Double subAngleY (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThreshold (get/set)
  Double searchRangeX (get/set)
  Double searchRangeY (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double peakSearchSpeedX (get/set)
  Double peakSearchSpeedY (get/set)
  UInt32 smoothingRangeX (get/set)
  UInt32 smoothingRangeY (get/set)
  Double centroidThresholdX (get/set)
  Double centroidThresholdY (get/set)
  Double convergentRangeX (get/set)
```

```
  Double convergentRangeY (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+FocusParameter
================================================================================================

Propiedades:
  ZMode zMode (get/set)
  UInt16 mainStageNumberX (get/set)
  UInt16 mainStageNumberY (get/set)
  UInt16 mainStageNumberZ (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleX (get/set)
  Double subAngleY (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThresholdXY (get/set)
  Double peakSearchThresholdZ (get/set)
  Double searchRangeX (get/set)
  Double searchRangeY (get/set)
  Double searchRangeZ (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchPitchZ (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double fieldSearchSpeedZ (get/set)
  Double peakSearchSpeedX (get/set)
  Double peakSearchSpeedY (get/set)
  Double peakSearchSpeedZ (get/set)
  UInt32 smoothingRangeX (get/set)
  UInt32 smoothingRangeY (get/set)
  UInt32 smoothingRangeZ (get/set)
  Double centroidThresholdX (get/set)
  Double centroidThresholdY (get/set)
  Double centroidThresholdZ (get/set)
  Double convergentRangeX (get/set)
  Double convergentRangeY (get/set)
  Double convergentRangeZ (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ICommonParameter
================================================================================================

Propiedades:
  AlignmentType alignmentType (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+IFlatParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberY (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleY (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThreshold (get/set)
  Double searchRangeY (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double peakSearchSpeedY (get/set)
  UInt32 smoothingRangeY (get/set)
  Double centroidThresholdY (get/set)
  Double convergentRangeY (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+IFocusParameter
================================================================================================

Propiedades:
  ZMode zMode (get/set)
  UInt16 mainStageNumberY (get/set)
  UInt16 mainStageNumberZ (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleY (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThresholdXY (get/set)
  Double peakSearchThresholdZ (get/set)
  Double searchRangeY (get/set)
  Double searchRangeZ (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
```

```
  Double fieldSearchPitchZ (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double fieldSearchSpeedZ (get/set)
  Double peakSearchSpeedY (get/set)
  Double peakSearchSpeedZ (get/set)
  UInt32 smoothingRangeY (get/set)
  UInt32 smoothingRangeZ (get/set)
  Double centroidThresholdY (get/set)
  Double centroidThresholdZ (get/set)
  Double convergentRangeY (get/set)
  Double convergentRangeZ (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ILoopBackParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberY (get/set)
  UInt16 mainStageNumberZ (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleY (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThresholdXY (get/set)
  Double peakSearchThresholdZ (get/set)
  Double searchRangeY (get/set)
  Double searchRangeZ (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchPitchZ (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double fieldSearchSpeedZ (get/set)
  Double peakSearchSpeedY (get/set)
  Double peakSearchSpeedZ (get/set)
  UInt32 smoothingRangeY (get/set)
  UInt32 smoothingRangeZ (get/set)
  Double centroidThresholdY (get/set)
  Double centroidThresholdZ (get/set)
  Double convergentRangeY (get/set)
  Double convergentRangeZ (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+IMMFRotationParameter
================================================================================================

Propiedades:
  UInt16 mmfStageNumber (get/set)
  Double mmfSpeed (get/set)
  Double mmfDistance (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ISharingParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberX (get/set)
  Double subAngleX (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Double searchRangeX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double peakSearchSpeedX (get/set)
  UInt32 smoothingRangeX (get/set)
  Double centroidThresholdX (get/set)
  Double convergentRangeX (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ISingleParameter
================================================================================================

Propiedades:
  ZMode zMode (get/set)
  UInt16 subStageNumberX (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThreshold (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ITwoChRotationParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberY (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleY (get/set)
  UInt16 rotateStageNumber (get/set)
  Double rotateSpeed (get/set)
  Int16 rotateDirection (get/set)
  Double coreDistance (get/set)
  UInt16 pmCh1 (get/set)
  UInt16 pmCh2 (get/set)
```

```
  UInt16 analogCh1 (get/set)
  UInt16 analogCh2 (get/set)
  Int16 pmInitRangeCh1 (get/set)
  Int16 pmInitRangeCh2 (get/set)
  Double fieldSearchThresholdCh1 (get/set)
  Double fieldSearchThresholdCh2 (get/set)
  Double peakSearchThresholdCh1 (get/set)
  Double peakSearchThresholdCh2 (get/set)
  Double searchRangeY (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double peakSearchSpeedY (get/set)
  UInt32 smoothingRangeY (get/set)
  Double centroidThresholdY (get/set)
  Double convergentRangeY (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+LoopBackParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberX (get/set)
  UInt16 mainStageNumberY (get/set)
  UInt16 mainStageNumberZ (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleX (get/set)
  Double subAngleY (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThresholdXY (get/set)
  Double peakSearchThresholdZ (get/set)
  Double searchRangeX (get/set)
  Double searchRangeY (get/set)
  Double searchRangeZ (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchPitchZ (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double fieldSearchSpeedZ (get/set)
  Double peakSearchSpeedX (get/set)
  Double peakSearchSpeedY (get/set)
  Double peakSearchSpeedZ (get/set)
  UInt32 smoothingRangeX (get/set)
  UInt32 smoothingRangeY (get/set)
  UInt32 smoothingRangeZ (get/set)
  Double centroidThresholdX (get/set)
  Double centroidThresholdY (get/set)
  Double centroidThresholdZ (get/set)
  Double convergentRangeX (get/set)
  Double convergentRangeY (get/set)
  Double convergentRangeZ (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+MMFRotationParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberX (get/set)
  UInt16 mainStageNumberY (get/set)
  UInt16 subStageNumberXY (get/set)
  Double subAngleX (get/set)
  Double subAngleY (get/set)
  UInt16 rotateStageNumber (get/set)
  Double rotateSpeed (get/set)
  Int16 rotateDirection (get/set)
  Double coreDistance (get/set)
  UInt16 mmfStageNumber (get/set)
  Double mmfSpeed (get/set)
  Double mmfDistance (get/set)
  UInt16 pmCh1 (get/set)
  UInt16 pmCh2 (get/set)
  UInt16 analogCh1 (get/set)
  UInt16 analogCh2 (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Int16 pmInitRangeCh1 (get/set)
  Int16 pmInitRangeCh2 (get/set)
  Double fieldSearchThresholdCh1 (get/set)
  Double fieldSearchThresholdCh2 (get/set)
  Double peakSearchThresholdCh1 (get/set)
  Double peakSearchThresholdCh2 (get/set)
  Double searchRangeX (get/set)
  Double searchRangeY (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double peakSearchSpeedX (get/set)
```

```
  Double peakSearchSpeedY (get/set)
  UInt32 smoothingRangeX (get/set)
  UInt32 smoothingRangeY (get/set)
  Double centroidThresholdX (get/set)
  Double centroidThresholdY (get/set)
  Double convergentRangeX (get/set)
  Double convergentRangeY (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ProfileData
================================================================================================

Campos:
  UInt16 mainAxisNumber
  UInt16 sub1AxisNumber
  UInt16 sub2AxisNumber
  UInt16 signalCh1Number
  UInt16 signalCh2Number
  UInt32 packetIndex
  UInt32 dataCount
  Double[] mainPositionList
  Double[] sub1PositionList
  Double[] sub2PositionList
  Double[] signalCh1List
  Double[] signalCh2List

================================================================================================
SurugaSeiki.Motion.Alignment+ProfileDataType
================================================================================================

Campos:
  Int32 value__
  static ProfileDataType FieldSearch
  static ProfileDataType PeakSearchX
  static ProfileDataType PeakSearchY
  static ProfileDataType PeakSearchZ
  static ProfileDataType PeakSearchXCh2
  static ProfileDataType PeakSearchYCh2

================================================================================================
SurugaSeiki.Motion.Alignment+SingleParameter
================================================================================================

Propiedades:
  ZMode zMode (get/set)
  UInt16 mainStageNumberX (get/set)
  UInt16 subStageNumberX (get/set)
  Double subAngleX (get/set)
  UInt16 pmCh (get/set)
  UInt16 analogCh (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Int16 pmInitRange (get/set)
  Double fieldSearchThreshold (get/set)
  Double peakSearchThreshold (get/set)
  Double searchRangeX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double peakSearchSpeedX (get/set)
  UInt32 smoothingRangeX (get/set)
  Double centroidThresholdX (get/set)
  Double convergentRangeX (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+Status
================================================================================================

Campos:
  Int32 value__
  static Status Stopping
  static Status Success
  static Status Aligning
  static Status FieldSearchRangeOver
  static Status ProfileDataOver
  static Status PeakSearchCountOver
  static Status PeakSearchRangeOver
  static Status InvalidParameter
  static Status ServoIsNotReady
  static Status ServoIsAlarm
  static Status StageOnLimit
  static Status VoltageLimit
  static Status PMRangeLimit
  static Status PMInitRangeChangeFail
  static Status PMDisconnected
  static Status RotationAdjustmentFail
  static Status InPositionFail
  static Status TorqueLimit

================================================================================================
SurugaSeiki.Motion.Alignment+TwoChRotationParameter
================================================================================================

Propiedades:
  UInt16 mainStageNumberX (get/set)
  UInt16 mainStageNumberY (get/set)
  UInt16 subStageNumberXY (get/set)
```

```
  Double subAngleX (get/set)
  Double subAngleY (get/set)
  UInt16 rotateStageNumber (get/set)
  Double rotateSpeed (get/set)
  Int16 rotateDirection (get/set)
  Double coreDistance (get/set)
  UInt16 pmCh1 (get/set)
  UInt16 pmCh2 (get/set)
  UInt16 analogCh1 (get/set)
  UInt16 analogCh2 (get/set)
  Boolean pmAutoRangeUpOn (get/set)
  Boolean pmInitRangeSettingOn (get/set)
  Int16 pmInitRangeCh1 (get/set)
  Int16 pmInitRangeCh2 (get/set)
  Double fieldSearchThresholdCh1 (get/set)
  Double fieldSearchThresholdCh2 (get/set)
  Double peakSearchThresholdCh1 (get/set)
  Double peakSearchThresholdCh2 (get/set)
  Double searchRangeX (get/set)
  Double searchRangeY (get/set)
  Double fieldSearchPitchX (get/set)
  Double fieldSearchPitchY (get/set)
  Double fieldSearchFirstPitchX (get/set)
  Double fieldSearchSpeedX (get/set)
  Double fieldSearchSpeedY (get/set)
  Double peakSearchSpeedX (get/set)
  Double peakSearchSpeedY (get/set)
  UInt32 smoothingRangeX (get/set)
  UInt32 smoothingRangeY (get/set)
  Double centroidThresholdX (get/set)
  Double centroidThresholdY (get/set)
  Double convergentRangeX (get/set)
  Double convergentRangeY (get/set)
  UInt16 comparisonCount (get/set)
  UInt16 maxRepeatCount (get/set)

================================================================================================
SurugaSeiki.Motion.Alignment+ZMode
================================================================================================

Campos:
  Int32 value__
  static ZMode Round
  static ZMode ForwardPlus
  static ZMode ForwardMinus
```

C.2 Clase AngleAdjustment

```
================================================================================================
SurugaSeiki.Motion.AngleAdjustment
================================================================================================

Constructores:
  AngleAdjustment(UInt16 id)

Metodos:
  ErrorCode Start()
  ErrorCode Stop()
  Status GetStatus()
  UInt32 GetErrorAxisID()
  AdjustingStatus GetAdjustingStatus()
  UInt32 GetProfilePacketSumIndex(ProfileDataType profileDataType)
  ProfileData RequestProfileData(ProfileDataType profileDataType, UInt32 index)
  ErrorCode SetParameter(AngleAdjustmentParameter parameter)
  ErrorCode SetParameter(AngleAdjustmentParameter parameter, RotationCenter rotationCenter)

Tipos anidados/enums:
  SurugaSeiki.Motion.AngleAdjustment+ErrorCode
  SurugaSeiki.Motion.AngleAdjustment+Status
  SurugaSeiki.Motion.AngleAdjustment+AdjustingStatus
  SurugaSeiki.Motion.AngleAdjustment+ProfileDataType
  SurugaSeiki.Motion.AngleAdjustment+AngleAdjustmentParameter
  SurugaSeiki.Motion.AngleAdjustment+ProfileData

================================================================================================
SurugaSeiki.Motion.AngleAdjustment+AdjustingStatus
================================================================================================

Campos:
  Int32 value__
  static AdjustingStatus NotAdjusting
  static AdjustingStatus Initializing
  static AdjustingStatus ContactingZ
  static AdjustingStatus AdjustingTx
  static AdjustingStatus AdjustingTy

================================================================================================
SurugaSeiki.Motion.AngleAdjustment+AngleAdjustmentParameter
================================================================================================

Campos:
  Double gap
  UInt16 signalChNumber
```

```
  Double signalLowerLimit
  UInt16 unlockDOutChNumber
  Boolean unlockDOutControlOn
  Boolean lockUnlockAdjustEnbale
  Double lockUnlockDifference
  UInt16 contactAxisNumber
  Double contactSearchRange
  Double contactSearchSpeed
  UInt32 contactSmoothing
  UInt32 contactSensitivity
  Double pushDistance
  UInt16 angleAxisNumberTx
  UInt16 angleAxisNumberTy
  Double angleSearchRangeTx
  Double angleSearchRangeTy
  Double angleSearchSpeedTx
  Double angleSearchSpeedTy
  UInt32 angleSmoothingTx
  UInt32 angleSmoothingTy
  UInt32 angleSensitivityTx
  UInt32 angleSensitivityTy
  UInt16 angleJudgeCountTx
  UInt16 angleJudgeCountTy
  Double angleConvergentRangeTx
  Double angleConvergentRangeTy
  UInt16 angleComparisonCount
  UInt16 angleMaxCount

================================================================================================
SurugaSeiki.Motion.AngleAdjustment+ErrorCode
================================================================================================

Campos:
  Int32 value__
  static ErrorCode None
  static ErrorCode Axis
  static ErrorCode Adjusting
  static ErrorCode Parameter

================================================================================================
SurugaSeiki.Motion.AngleAdjustment+ProfileData
================================================================================================

Campos:
  UInt16 mainAxisNumber
  UInt16 sub1AxisNumber
  UInt16 sub2AxisNumber
  UInt16 signalCh1Number
  UInt16 signalCh2Number
  UInt32 packetIndex
  UInt32 dataCount
  Double[] mainPositionList
  Double[] sub1PositionList
  Double[] sub2PositionList
  Double[] signalCh1List
  Double[] signalCh2List

================================================================================================
SurugaSeiki.Motion.AngleAdjustment+ProfileDataType
================================================================================================

Campos:
  Int32 value__
  static ProfileDataType ContactZ
  static ProfileDataType AdjustmentTx
  static ProfileDataType AdjustmentTy

================================================================================================
SurugaSeiki.Motion.AngleAdjustment+Status
================================================================================================

Campos:
  Int32 value__
  static Status Stopping
  static Status Success
  static Status Adjusting
  static Status ProfileDataOver
  static Status InvalidParameter
  static Status ServoIsNotReady
  static Status ServoIsAlarm
  static Status StageOnLimit
  static Status SignalLowerLimit
  static Status CouldnotContact
  static Status AdjustCountOver
  static Status AngleAdjustRnageOver
  static Status LostContact
```

C.3 Clase Axis2D

```
================================================================================================
SurugaSeiki.Motion.Axis2D
================================================================================================
```

```
Constructores:
  Axis2D(UInt16 mainStageNumber, UInt16 subStageNumber)

Metodos:
  ErrorCode SetAxisNumber(UInt16 mainStageNumber, UInt16 subStageNumber)
  UInt16[] GetAxisNumber()
  ErrorCode MoveAbsolute(Point targetPosition)
  ErrorCode MoveAbsolute(Double mainTargetPosition, Double subTargetPosition)
  ErrorCode MoveRelative(Point targetDistance)
  ErrorCode MoveRelative(Double mainDistance, Double subDistance)
  ErrorCode MoveAbsoluteAngle(Double mainTargetPosition, Double subAngle)
  ErrorCode MoveRelativeAngle(Double mainDistance, Double subAngle)
  ErrorCode Stop()
  Point GetTargetPosition()
  Point GetActualPosition()
  Boolean IsMoving()
  ErrorCode SetSpeed(Double speed)
  ErrorCode SetAccelRate(Double rate)
  ErrorCode SetDecelRate(Double rate)
  ErrorCode GetErrorCode()
  Status GetStatus()

Tipos anidados/enums:
  SurugaSeiki.Motion.Axis2D+Point

================================================================================================
SurugaSeiki.Motion.Axis2D+Point
================================================================================================

Campos:
  Double X
  Double Y
```

C.4 Clase Axis3D

```
================================================================================================
SurugaSeiki.Motion.Axis3D
================================================================================================

Constructores:
  Axis3D(UInt16 mainStageNumberX, UInt16 mainStageNumberY, UInt16 mainStageNumberZ)

Metodos:
  ErrorCode SetAxisNumber(UInt16 mainStageNumberX, UInt16 mainStageNumberY, UInt16 mainStageNumberZ)
  UInt16[] GetAxisNumber()
  ErrorCode MoveAbsolute(Point3D targetPosition)
  ErrorCode MoveAbsolute(Double targetPositionX, Double targetPositionY, Double targetPositionZ)
  ErrorCode MoveAbsolute(Point3D targetPosition, UInt16 dOutId, Int16 dOutOnDelayTime, Int16 dOutOffDelayTime)
  ErrorCode MoveAbsolute(Double targetPositionX, Double targetPositionY, Double targetPositionZ, UInt16 dOutId, Int16 dOutOnDelayTime, Int16
      dOutOffDelayTime)
  ErrorCode MoveRelative(Point3D targetDistance)
  ErrorCode MoveRelative(Double targetDistanceX, Double targetDistanceY, Double targetDistanceZ)
  ErrorCode MoveRelative(Point3D targetDistance, UInt16 dOutId, Int16 dOutOnDelayTime, Int16 dOutOffDelayTime)
  ErrorCode MoveRelative(Double targetDistanceX, Double targetDistanceY, Double targetDistanceZ, UInt16 dOutId, Int16 dOutOnDelayTime, Int16
      dOutOffDelayTime)
  ErrorCode Stop()
  Point3D GetTargetPosition()
  Point3D GetActualPosition()
  Boolean IsMoving()
  ErrorCode SetSpeed(Double speed)
  ErrorCode SetAccelRate(Double accelRate)
  ErrorCode SetDecelRate(Double decelRate)
  ErrorCode SetStageNumber(UInt16 mainStageNumberX, UInt16 mainStageNumberY, UInt16 mainStageNumberZ)
  ErrorCode SetSubStageNumber(UInt16 subStageNumberX, UInt16 subStageNumberY, UInt16 subStageNumberZ)
  ErrorCode SetRotationCenter(Double rotationOffsetX, Double rotationOffsetY, Double rotationOffsetZ)
  ErrorCode GetErrorCode()
  Status GetStatus()

Tipos anidados/enums:
  SurugaSeiki.Motion.Axis3D+Point3D
  SurugaSeiki.Motion.Axis3D+RotationCenter

================================================================================================
SurugaSeiki.Motion.Axis3D+Point3D
================================================================================================

Campos:
  Double X
  Double Y
  Double Z

================================================================================================
SurugaSeiki.Motion.Axis3D+RotationCenter
================================================================================================

Campos:
  UInt16 mainStageNumberX
  UInt16 mainStageNumberY
  UInt16 mainStageNumberZ
  UInt16 subStageNumberX
  UInt16 subStageNumberY
  UInt16 subStageNumberZ
  Double offsetX
```

```
  Double offsetY
  Double offsetZ
```

C.5 Clase AxisComponents

```
================================================================================================
SurugaSeiki.Motion.AxisComponents
================================================================================================

Constructores:
  AxisComponents(UInt16 axisNumberm)

Metodos:
  ErrorCode MoveAbsolute(Double position)
  ErrorCode MoveRelative(Double distance)
  ErrorCode ReturnOrigin()
  ErrorCode Stop()
  ErrorCode TurnOnServo()
  ErrorCode TurnOffServo()
  ErrorCode ResetError()
  Double GetTargetPosition()
  Double GetActualPosition()
  Boolean IsServoOn()
  Boolean IsServoAlarm()
  Boolean IsMoving()
  Boolean IsOriginSignalOn()
  Boolean IsCwLimitSignalOn()
  Boolean IsCcwLimitSignalOn()
  Boolean IsCwTorqueLimitOn()
  Boolean IsCcwTorqueLimitOn()
  Boolean IsOriginReturnSucceeded()
  ErrorCode SetAccelRate(Double rate)
  ErrorCode SetDecelRate(Double rate)
  ErrorCode SetSineMotion(Boolean sineMotion)
  ErrorCode SetFulcrumSpan(Double span)
  ErrorCode SetEncoderStepLength(Double length)
  ErrorCode SetMaxSpeed(Double speed)
  ErrorCode SetCWTorqueLimit(Double cWlimit)
  ErrorCode SetCCWTorqueLimit(Double cCWlimit)
  ErrorCode GetErrorCode()
  Status GetStatus()

Tipos anidados/enums:
  SurugaSeiki.Motion.AxisComponents+ErrorCode
  SurugaSeiki.Motion.AxisComponents+Status

================================================================================================
SurugaSeiki.Motion.AxisComponents+ErrorCode
================================================================================================

Campos:
  Int32 value__
  static ErrorCode None
  static ErrorCode ServoAlarm
  static ErrorCode ServoOff
  static ErrorCode Moving
  static ErrorCode NotConnected
  static ErrorCode Limit
  static ErrorCode Origin
  static ErrorCode Parameter
  static ErrorCode Torque

================================================================================================
SurugaSeiki.Motion.AxisComponents+Status
================================================================================================

Campos:
  Int32 value__
  static Status ServoAlarm
  static Status ServoOff
  static Status Stopping
  static Status InPosition
  static Status Moving
  static Status NotConnected
  static Status CwLimit
  static Status CcwLimit
  static Status CwTorqueLimit
  static Status CcwTorqueLimit
```

C.6 Clase ErrorLog

```
================================================================================================
SurugaSeiki.Motion.ErrorLog
================================================================================================
```

C.7 Clase IO

```
================================================================================================
SurugaSeiki.Motion.IO
================================================================================================

Constructores:
  IO()

Metodos:
  Boolean GetPortState(DigitalIOType iOType, UInt16 id)
  Void SetPortState(UInt16 id, Boolean portState)
  Void SetPulseOutput(UInt16 id, UInt16 delayTime, UInt16 highWidth, UInt16 lowWidth, Int16 repetitionCount)
  Double GetAnalogValue(AnalogIOType iOType, UInt16 id)
  Void SetAnalogValue(UInt16 id, Double setValue)

Campos:
  static Int16 RepetitionCountForever

Tipos anidados/enums:
  SurugaSeiki.Motion.IO+DigitalIOType
  SurugaSeiki.Motion.IO+AnalogIOType

================================================================================================
SurugaSeiki.Motion.IO+AnalogIOType
================================================================================================

Campos:
  Int32 value__
  static AnalogIOType Input
  static AnalogIOType Output

================================================================================================
SurugaSeiki.Motion.IO+DOutControlParameterToAlignmentSystem
================================================================================================

================================================================================================
SurugaSeiki.Motion.IO+DigitalIOType
================================================================================================

Campos:
  Int32 value__
  static DigitalIOType Input
  static DigitalIOType Output
```

C.8 Clase PowerMeter

```
================================================================================================
SurugaSeiki.Motion.PowerMeter
================================================================================================

Constructores:
  PowerMeter()

Metodos:
  Double GetPower(UInt16 ch)
  AllowedRange GetRange(UInt16 ch)
  Double GetWavelength(UInt16 ch)
  ErrorCode SetRange(UInt16 ch, AllowedRange range)
  ErrorCode SetWaveLength(UInt16 ch, Double waveLength)
  ErrorCode PauseCommunication(UInt16 ch, Boolean isEnabled)

Tipos anidados/enums:
  SurugaSeiki.Motion.PowerMeter+ErrorCode
  SurugaSeiki.Motion.PowerMeter+AllowedRange

================================================================================================
SurugaSeiki.Motion.PowerMeter+AllowedRange
================================================================================================

Campos:
  Int16 value__
  static AllowedRange Minus60
  static AllowedRange Minus50
  static AllowedRange Minus40
  static AllowedRange Minus30
  static AllowedRange Minus20
  static AllowedRange Minus10
  static AllowedRange Zero
  static AllowedRange Plus10
  static AllowedRange Plus20
  static AllowedRange Auto
  static AllowedRange Invalid

================================================================================================
SurugaSeiki.Motion.PowerMeter+ErrorCode
================================================================================================

Campos:
  Int32 value__
  static ErrorCode None
  static ErrorCode ChannelOutOfRange
```

```
  static ErrorCode Invalid
```

C.9 Clase System

```
================================================================================================
SurugaSeiki.Motion.System
================================================================================================

Propiedades:
  UInt16 MaxAxisStageNumber (get/set)
  UInt16 MaxAnalogInputChNumber (get/set)
  UInt16 MaxAlignmentNumber (get/set)
  UInt16 MaxAngleAdjustmentNumber (get/set)
  UInt16 MaxDInChNumber (get/set)
  UInt16 MaxDOutChNumber (get/set)
  UInt16 MaxAnalogOutputChNumber (get/set)
  UInt16 MaxPowerMeterChNumber (get/set)
  System Instance (get)
  Boolean Connected (get)
  String DllVersion (get)
  String SystemVersion (get)

Metodos:
  Boolean IsError()
  ErrorCode GetCurrentErrorCode()
  Boolean IsEmergencyAsserted()
  Void SetAddress(String addr)
  Void DisconnectAds()

Campos:
  static UInt16 MaxProfileNumber
  static UInt32 ProfileDataPacketMaxDataNumber

Tipos anidados/enums:
  SurugaSeiki.Motion.System+ErrorCode

================================================================================================
SurugaSeiki.Motion.System+AlignmentChParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentCommand
================================================================================================

Campos:
  Int32 value__
  static AlignmentCommand Null
  static AlignmentCommand StartSingle
  static AlignmentCommand StartFlat
  static AlignmentCommand StartFocus
  static AlignmentCommand StartTwoChRotation
  static AlignmentCommand StartMMFRotation
  static AlignmentCommand Stop
  static AlignmentCommand SetSingle
  static AlignmentCommand SetFlat
  static AlignmentCommand SetFocus
  static AlignmentCommand SetTwoChRotation
  static AlignmentCommand SetMMFRotation
  static AlignmentCommand StartLoopBack
  static AlignmentCommand SetLoopBack

================================================================================================
SurugaSeiki.Motion.System+AlignmentMainAxisParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentPowerMeterInfo
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AlignmentStatusInfo
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AnalogOutputControlCommand
================================================================================================

Campos:
  Int32 value__
  static AnalogOutputControlCommand Null
  static AnalogOutputControlCommand SetAnalogOutput

================================================================================================
SurugaSeiki.Motion.System+AngleAdjustmentCommand
================================================================================================

Campos:
  Int32 value__
  static AngleAdjustmentCommand Null
```

```
  static AngleAdjustmentCommand SetParameter
  static AngleAdjustmentCommand Start
  static AngleAdjustmentCommand Stop

================================================================================================
SurugaSeiki.Motion.System+AngleAdjustmentParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AngleAdjustmentStatusInfo
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+Axis2DCommand
================================================================================================

Campos:
  Int32 value__
  static Axis2DCommand Null
  static Axis2DCommand MoveAbsolute
  static Axis2DCommand MoveRelative
  static Axis2DCommand MoveAbsoluteAngle
  static Axis2DCommand MoveRelativeAngle
  static Axis2DCommand Stop
  static Axis2DCommand SetCompositionSpeed
  static Axis2DCommand SetAccelRate
  static Axis2DCommand SetDecelRate
  static Axis2DCommand SetStageNumber

================================================================================================
SurugaSeiki.Motion.System+Axis2DParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AxisCommandStruct
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+AxisComponentCommand
================================================================================================

Campos:
  Int32 value__
  static AxisComponentCommand Null
  static AxisComponentCommand MoveAbsolute
  static AxisComponentCommand MoveRelative
  static AxisComponentCommand ReturnOrigin
  static AxisComponentCommand Stop
  static AxisComponentCommand TurnOnServo
  static AxisComponentCommand TurnOffServo
  static AxisComponentCommand ResetError
  static AxisComponentCommand SetMaxSpeed
  static AxisComponentCommand SetAccelRate
  static AxisComponentCommand SetDecelRate
  static AxisComponentCommand SetSineMotion
  static AxisComponentCommand SetFulcrumSpan
  static AxisComponentCommand SetEncoderStepLength
  static AxisComponentCommand SetCWTorquelimit
  static AxisComponentCommand SetCCWTorquelimit

================================================================================================
SurugaSeiki.Motion.System+AxisControlType
================================================================================================

Campos:
  Int32 value__
  static AxisControlType AxisComponents
  static AxisControlType Axis2D
  static AxisControlType Axis3D

================================================================================================
SurugaSeiki.Motion.System+AxisMethod
================================================================================================

Campos:
  Int32 value__
  static AxisMethod Null
  static AxisMethod MoveAbsolute
  static AxisMethod MoveRelative
  static AxisMethod MoveAbsoluteAngle
  static AxisMethod MoveRelativeAngle
  static AxisMethod Stop
  static AxisMethod SetCompositionSpeed
  static AxisMethod SetAccelRate
  static AxisMethod SetDecelRate
  static AxisMethod SetStageNumber
  static AxisMethod SetSubStageNumber
  static AxisMethod SetRotationCenter
  static AxisMethod SetFixedAxisOffset

================================================================================================
SurugaSeiki.Motion.System+AxisStatus
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+DOutControlCommand
================================================================================================

Campos:
```

```
  Int32 value__
  static DOutControlCommand Null
  static DOutControlCommand SetDout
  static DOutControlCommand SetPulseOutput

================================================================================================
SurugaSeiki.Motion.System+DeviceControlCommand
================================================================================================

Campos:
  Int32 value__
  static DeviceControlCommand Null
  static DeviceControlCommand SetWaveLength

================================================================================================
SurugaSeiki.Motion.System+DeviceStatusMap
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+ErrorCode
================================================================================================

Campos:
  Int32 value__
  static ErrorCode NoError
  static ErrorCode ConnectionLost
  static ErrorCode VersionError
  static ErrorCode MaxNumberOfErrorCode

================================================================================================
SurugaSeiki.Motion.System+Point2d
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+PowerMeterControlCommand
================================================================================================

Campos:
  Int32 value__
  static PowerMeterControlCommand None
  static PowerMeterControlCommand SetRange
  static PowerMeterControlCommand SetWaveLength
  static PowerMeterControlCommand SetCommunicationPaused

================================================================================================
SurugaSeiki.Motion.System+ProfileCommand
================================================================================================

Campos:
  Int32 value__
  static ProfileCommand Null
  static ProfileCommand SetParameter
  static ProfileCommand Start
  static ProfileCommand Stop

================================================================================================
SurugaSeiki.Motion.System+ProfileParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+RotationCenterParam
================================================================================================

================================================================================================
SurugaSeiki.Motion.System+SystemInfo
================================================================================================
```

Bibliography